

POLITECHNIKA RZESZOWSKA
IM. IGNACEGO ŁUKASIEWICZA

Wydział Elektrotechniki i Informatyki

Katedra Informatyki i Automatyki

Sztuczna inteligencja

Projekt

Klasyfikacja danych ze zbioru „Wine”, z wykorzystaniem sztucznej sieci neuronowej (2 warstwy ukryte) uczonej metodą wstecznej propagacji błędów z przyspieszeniem uczenia metodą adaptacyjnego współczynnika uczenia oraz metodą momentum.

Kierunek: II EF-DI

Imię i nazwisko: Jakub Przystasz

Numer indeksu: 163998

Numer grupy: P6

Prowadzący:
mgr inż. Dawid Kalandyk

Rzeszów 2021

Spis treści

1	Wstęp	3
1.1	Opis problemu	3
1.2	Opis danych uczących	4
1.2.1	Badanie zbioru danych pod kątem klasyfikacji	6
1.2.2	Przygotowanie danych do procesu uczenia	19
1.3	Zagadnienia teoretyczne	20
1.3.1	Zasada działania oraz budowa sztucznego neuronu	20
1.3.2	Pojęcie warstwy neuronów, oraz sieci neuronowej	23
1.3.3	Proces nauki sztucznej sieci neuronowej	26
1.3.4	Algorytm wstecznej propagacji błędu	28
1.3.5	Adaptacyjny współczynnik uczenia	33
1.3.6	Metoda momentum	34
2	Implementacja sieci neuronowej	35
3	Eksperymenty	44
3.1	Eksperyment I - Badanie wpływu hiperparametrów na szybkość uczenia sieci	44
3.1.1	Badanie wpływu architektury sieci (hiperparametry S1, S2)	47
3.1.2	Badanie wpływu momentum constans	51
3.1.3	Badanie wpływu learning rate	52
3.1.4	Badanie wpływu współczynników learning rate accelerate, decelerate	54
3.1.5	Wnioski	59
3.2	Eksperyment II - Zjawisko przeuczania sieci	60
3.3	Eksperyment III - Metody aktualizacji wag	64
3.3.1	Porównanie metod aktualizacji wag	65
3.3.2	Porównanie metod inicjalizacji wag oraz biasów	66
3.3.3	Rozszerzenie metody momentum	69
4	Wnioski	72

1 Wstęp

1.1 Opis problemu

Celem projektu jest realizacja sieci neuronowej uczonej za pomocą algorytmu wstecznej propagacji błędów z przyspieszeniem metodą adaptacyjnego współczynnika uczenia oraz metodą momentum. Na potrzeby realizacji projektu zaimplementowano bibliotekę w języku C++ pozwalającą na realizację sieci neuronowej uczonej algorytmem wstecznej propagacji błędów, wraz z opcjonalnym rozszerzeniem metody uczenia o adaptacyjny współczynnik uczenia oraz metodę momentum. W ramach projektu zbadano wpływ parametrów sieci na szybkość uczenia:

- Ilość neuronów I warstwy ukrytej (S1)
- Ilość neuronów II warstwy ukrytej (S2)
- Wartość przyspieszenia współczynnika uczenia (Learning Rate Accelerate)
- Wartość spowolnienia współczynnika uczenia (Learning Rate Decelerate)
- Wartość współczynnika momentum constans

Jako zbiór danych uczących wykorzystano zbiór „Wine”[1].

1.2 Opis danych uczących

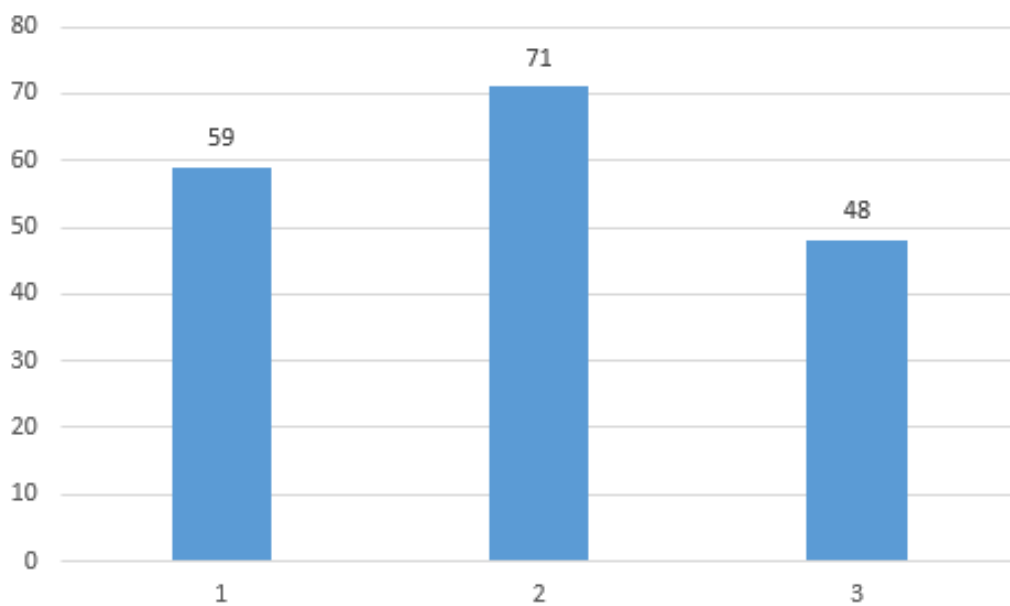
Zbiór danych „Wine”[1] zawiera 178 rekordów, każdy po 14 atrybutów. Dane są wynikiem analizy chemicznej win uprawianych w tym samym regionie Włoch, ale pochodzących z trzech różnych odmian. Badany zestaw danych nie zawiera niekompletnych rekordów, oraz wartości niepoprawnych. W zbiorze danych pierwszy atrybut określa rodzaj wina - opisany liczbą całkowitą $\{1, 2, 3\}$, natomiast pozostałe 13 atrybutów to:

- (Alcohol) - Procentowa zawartość alkoholu (wartość ciągła)
- (Malic acid) - Procentowa zawartość kwasu jabłkowego (wartość ciągła)
- (Ash) - Zawartość popiołu (wartość ciągła)
- (Alkalinity of ash) - Zasadowość popiołu (wartość ciągła)
- (Mg) - Zawartość magnezu (wartość ciągła)
- (Total phenols) - Zawartość fenoli (wartość ciągła)
- (Flavonoids) - Flawonoidy (wartość ciągła)
- (Non Flavonoid phenols) - Fenole nieflawonoidowe (wartość ciągła)
- (Proanthocyanins) - Proantocyjanidyny (wartość ciągła)
- (Colour intensity) - Intensywność barwy (wartość ciągła)
- (Hue) - Odcień (wartość ciągła)
- (OD280/OD315) (wartość ciągła)
- (Proline) - Prolina (wartość dyskretna)

Tabela 1 zawiera zestawienie statystyk dla każdego z atrybutów badanego zbioru, natomiast rysunek 1 prezentuje rozkład poszczególnych klas w zbiorze.

Atrybut	Średnia	Odch. std.	Min.	Q1	Mediana	Q3	Mak.
Alcohol	13.00	0.81	11.	12.	13.	14.	15.
Malic acid	2.	1.	0.74	1.60	2.	3.	5.80
Ash	2.	0.27	1.	2.	2.	3.	3.
Alkalinity of ash	19.	3.	10.60	17.20	19.50	21.50	30.00
Mg	100.	14.	70	88	98	107	162
Total phenols	2.30	0.63	0.98	2.	2.	2.80	4.
Flavonoids	2.	1.00	0.34	1.	2.	3.	5.
Non Flavonoid phenols	0.36	0.12	0.13	0.27	0.34	0.44	0.66
Proanthocyanins	2.	0.57	0.41	1.	2.	2.	4.
Colour intensity	5.	2.	1.	3.	5.	6.20	13.00
Hue	0.96	0.23	0.48	0.78	0.97	1.	2.
OD280/OD315 of diluted wines	3.	0.71	1.	2.	3.	3.	4.00
Proline	747	315	278	501	674	985	1680

Tabela 1: Statystyka atrybutów zbioru „Wine” [2]

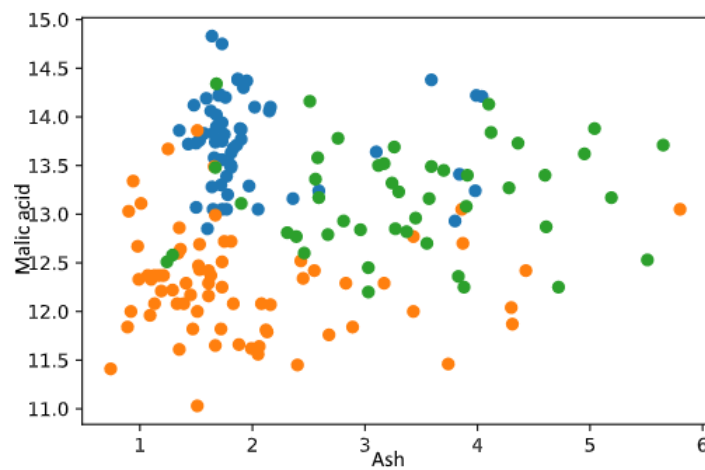


Rysunek 1: Rozkład klas w zbiorze „Wine”

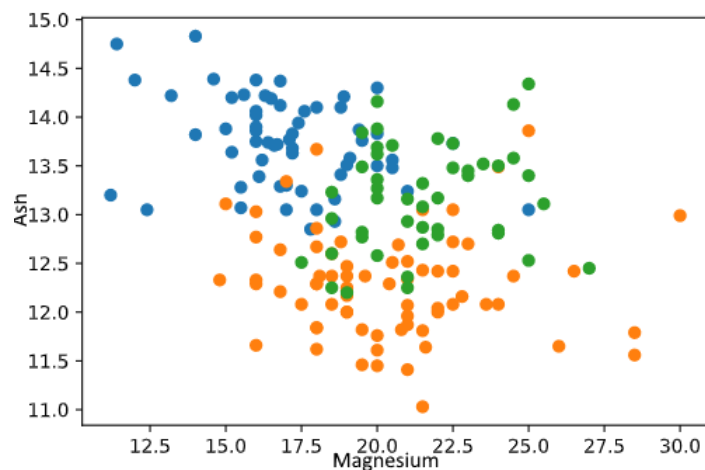
1.2.1 Badanie zbioru danych pod kątem klasyfikacji

Rysunki 2, oraz 3 zawierają zależność dwóch atrybutów wraz z rozróżnieniem na klasy:

- kolor niebieski - oznacza przynależność rekordu do klasy oznaczonej liczbą 1
- kolor pomarańczony - przynależność do klasy 2
- kolor zielony - przynależność rekordu do klasy 3

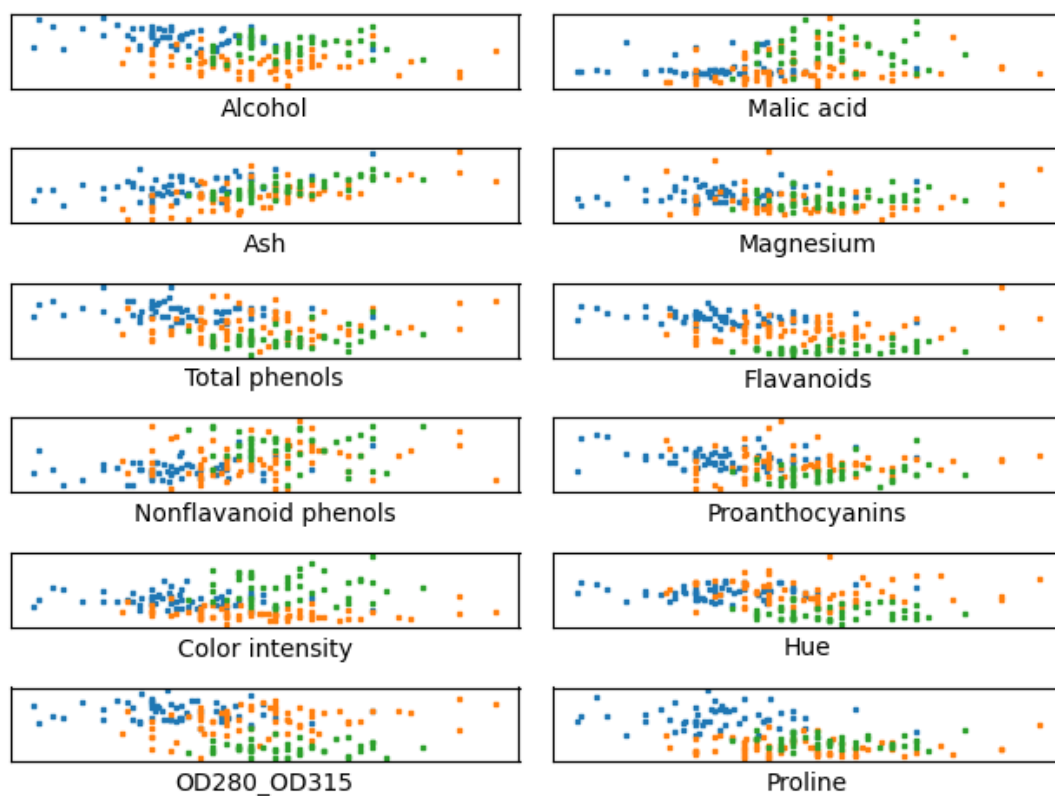


Rysunek 2: Zależność atrybutu 'Ash' od 'Malic acid'

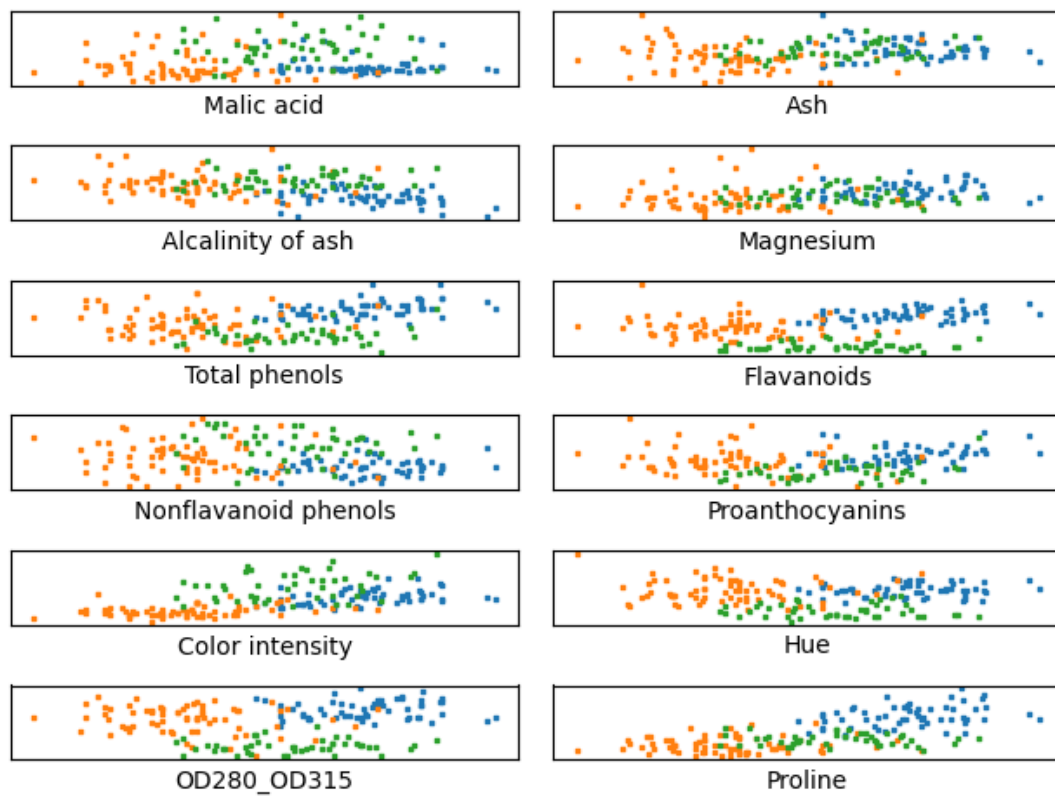


Rysunek 3: Zależność atrybutu 'Magnesium' od 'Ash'

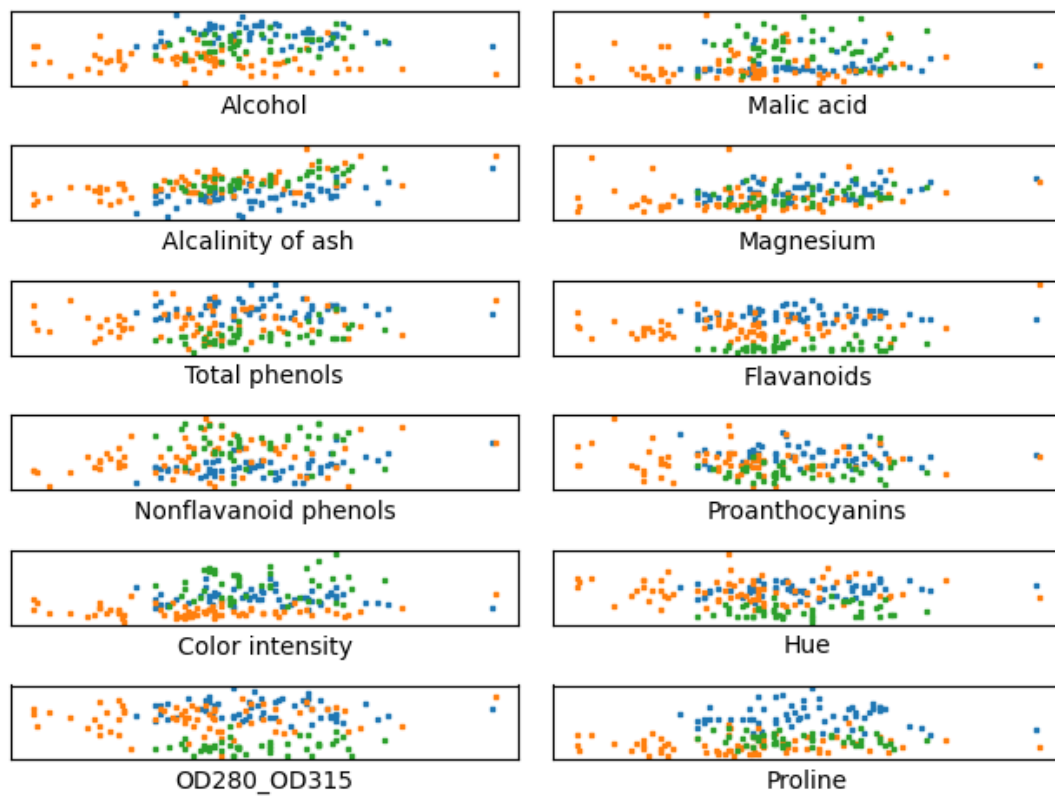
Na zamieszczonych poniżej wykresach pokazano zależność zmiennych we wszystkich kombinacjach par atrybutów. Oś rzędnych jest opatrzona podpisem zmiennej pod wykresem, oś odciętych w każdym z wykresów dotyczy zmiennej podpisanej pod rysunkiem.



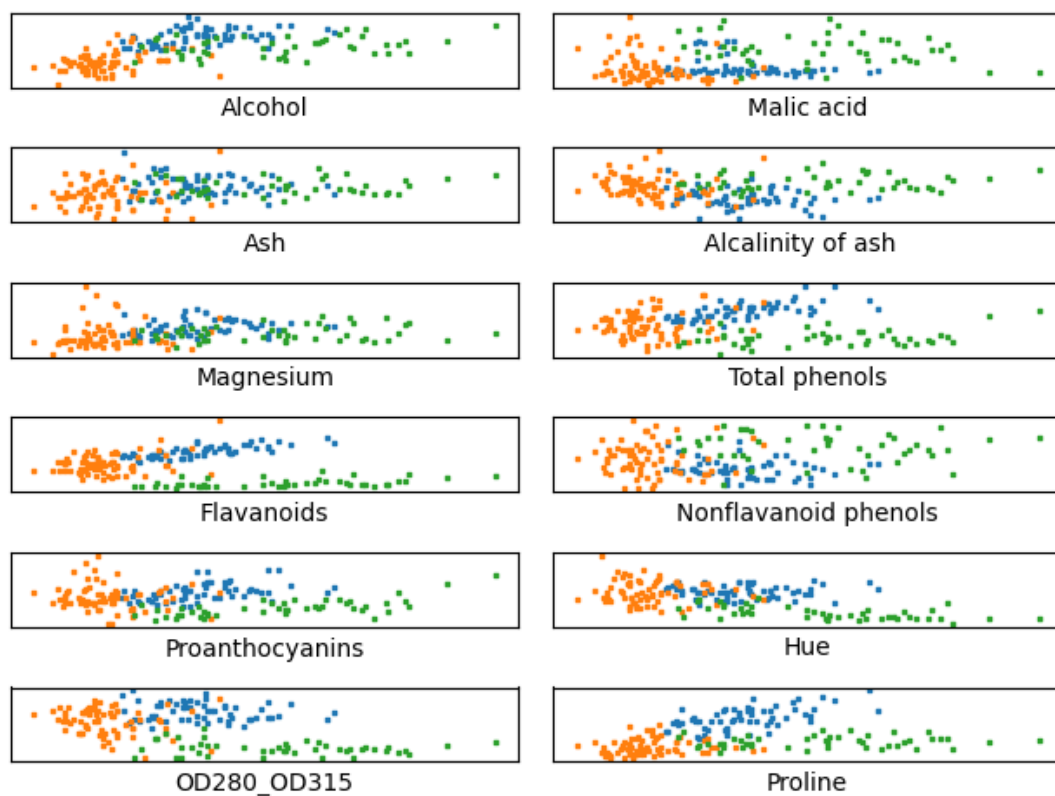
Rysunek 4: Zależność atrybutu 'Alcalinity'



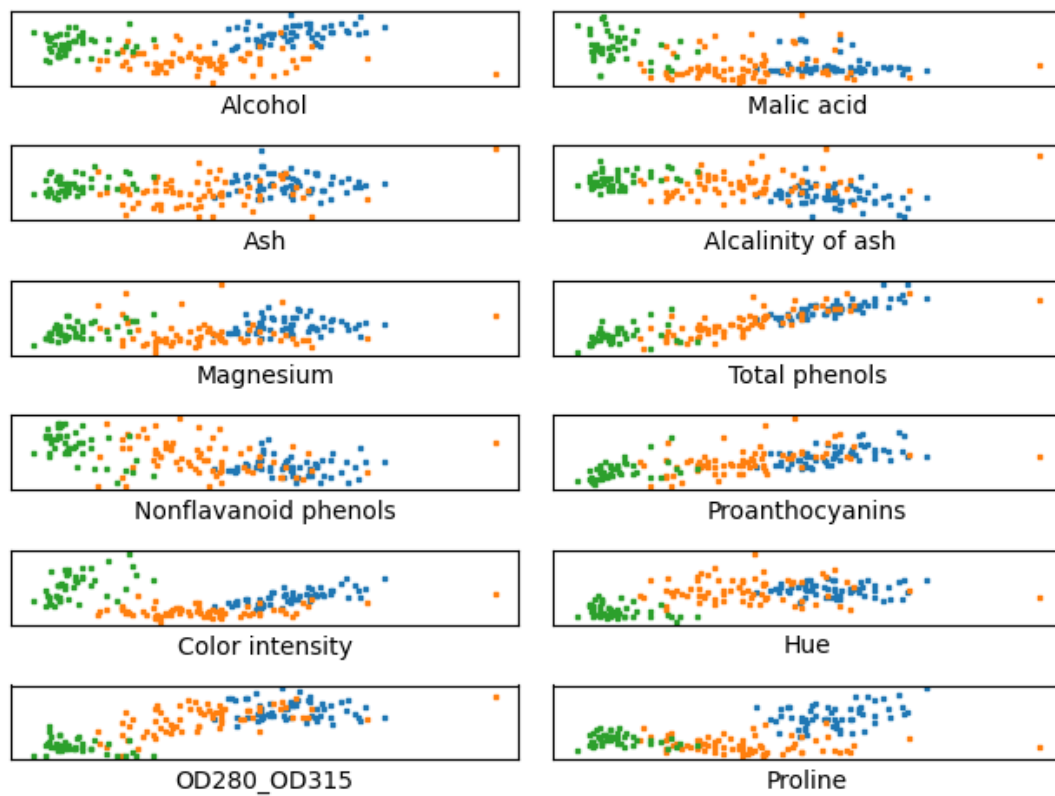
Rysunek 5: Zależność atrybutu 'Alcohol'



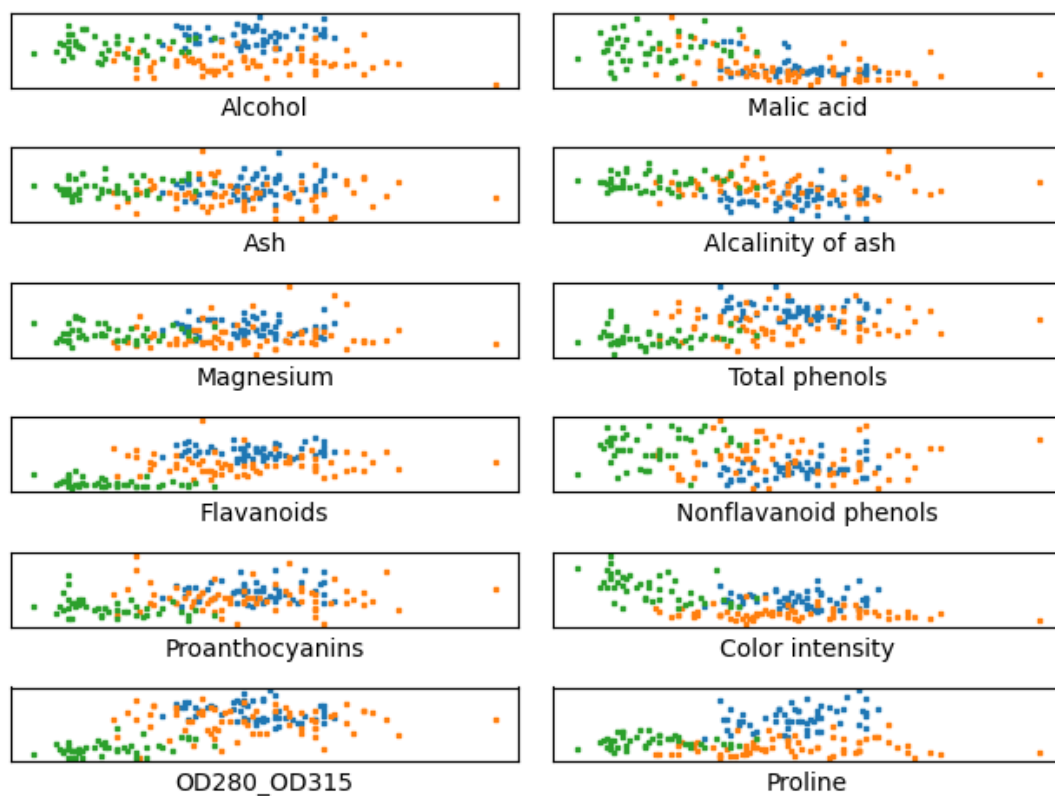
Rysunek 6: Zależność atrybutu 'Ash'



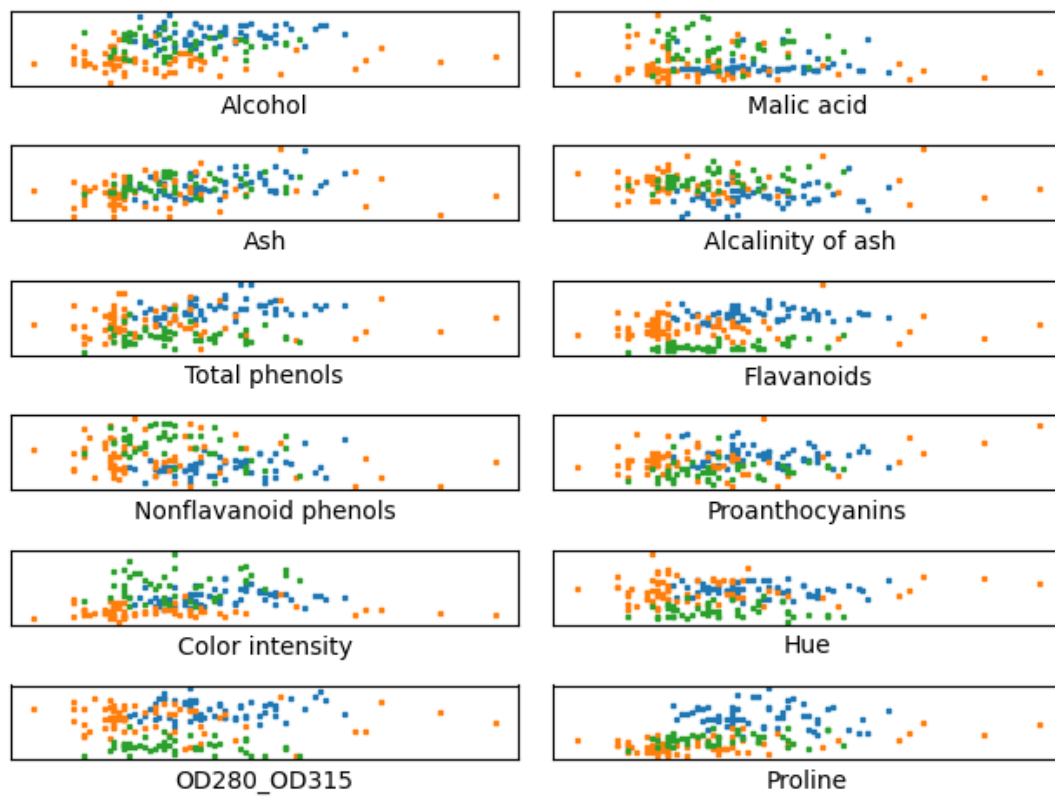
Rysunek 7: Zależność atrybutu 'Color intensity'



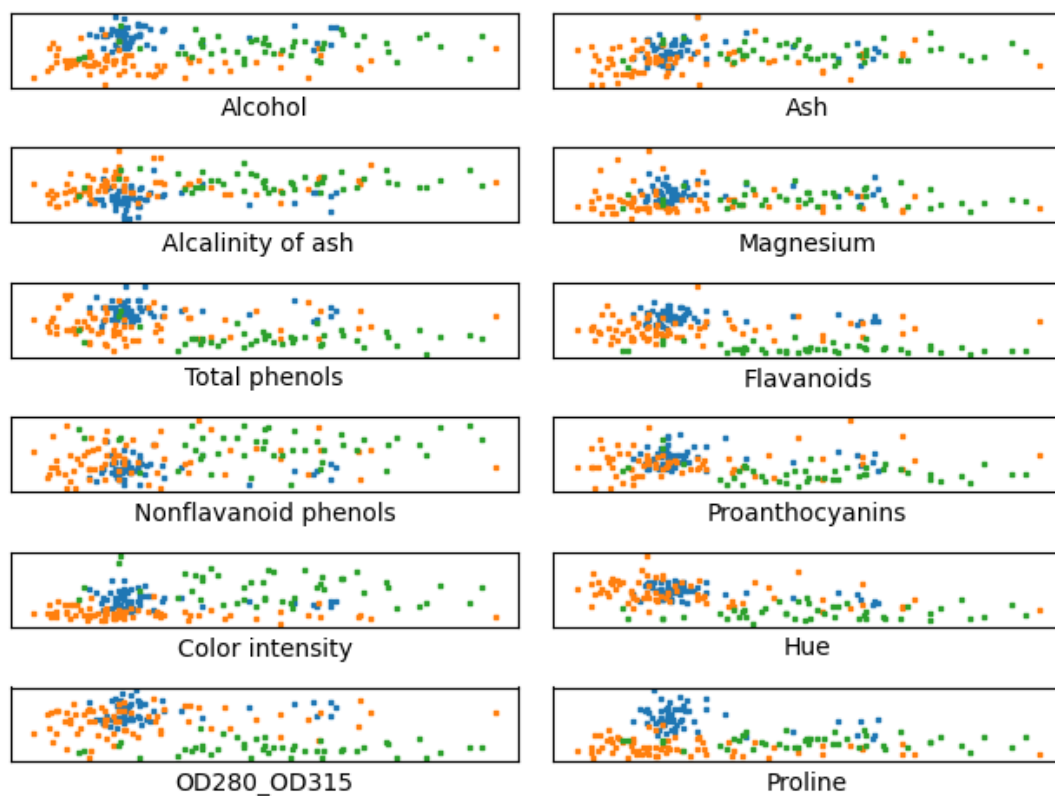
Rysunek 8: Zależność atrybutu 'Flavonoids'



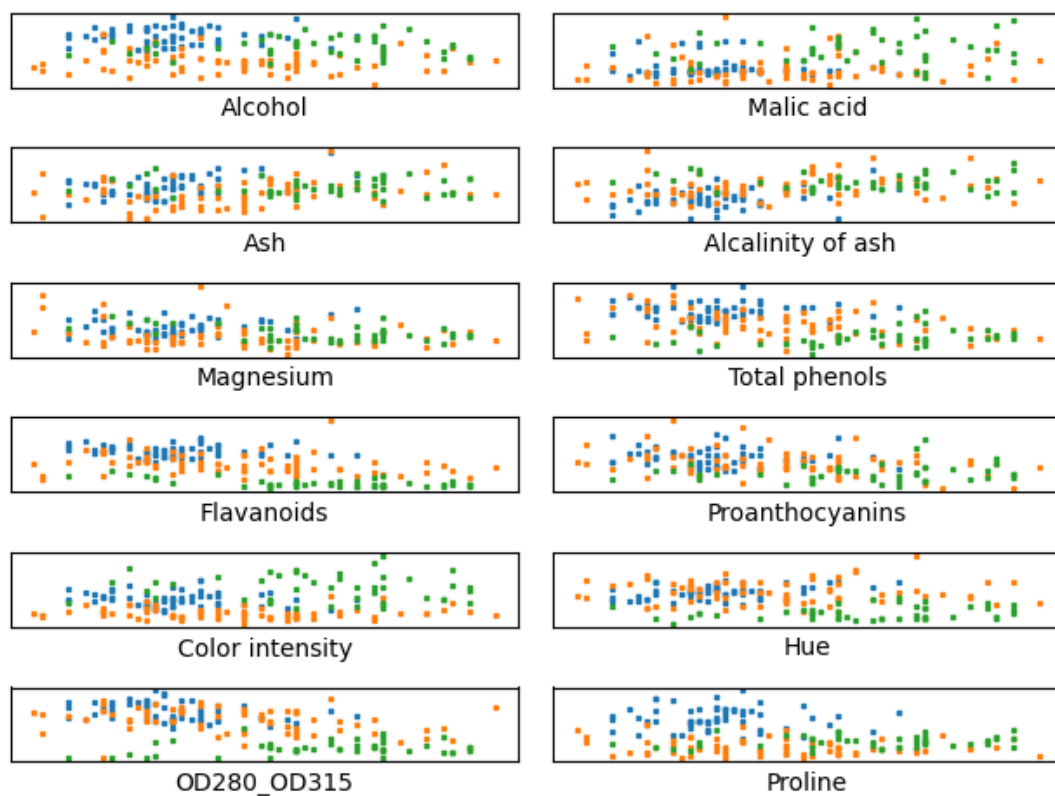
Rysunek 9: Zależność atrybutu 'Hue'



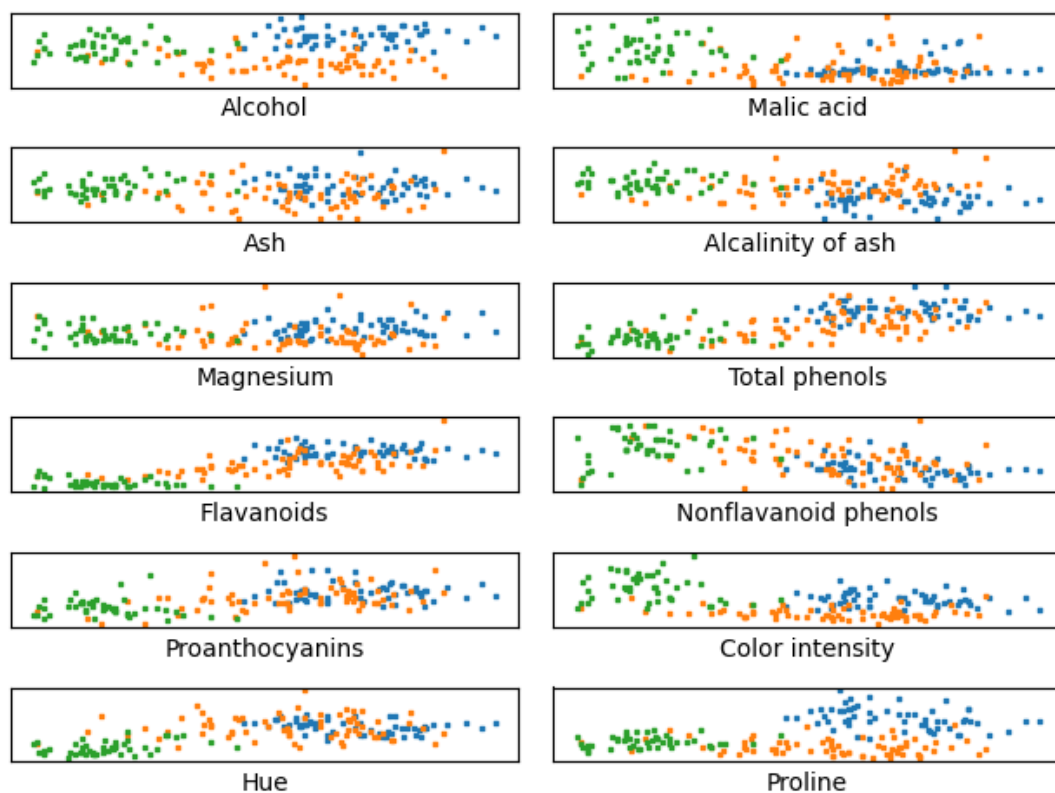
Rysunek 10: Zależność atrybutu 'Magnesium'



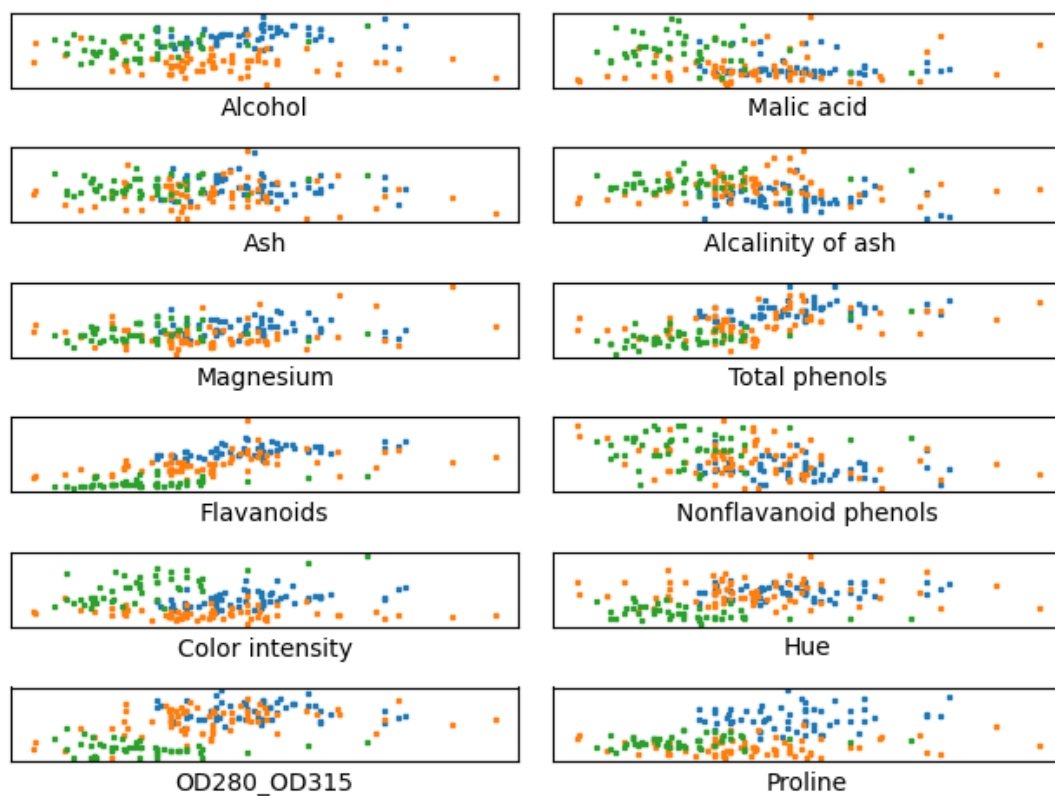
Rysunek 11: Zależność atrybutu 'Malic acid'



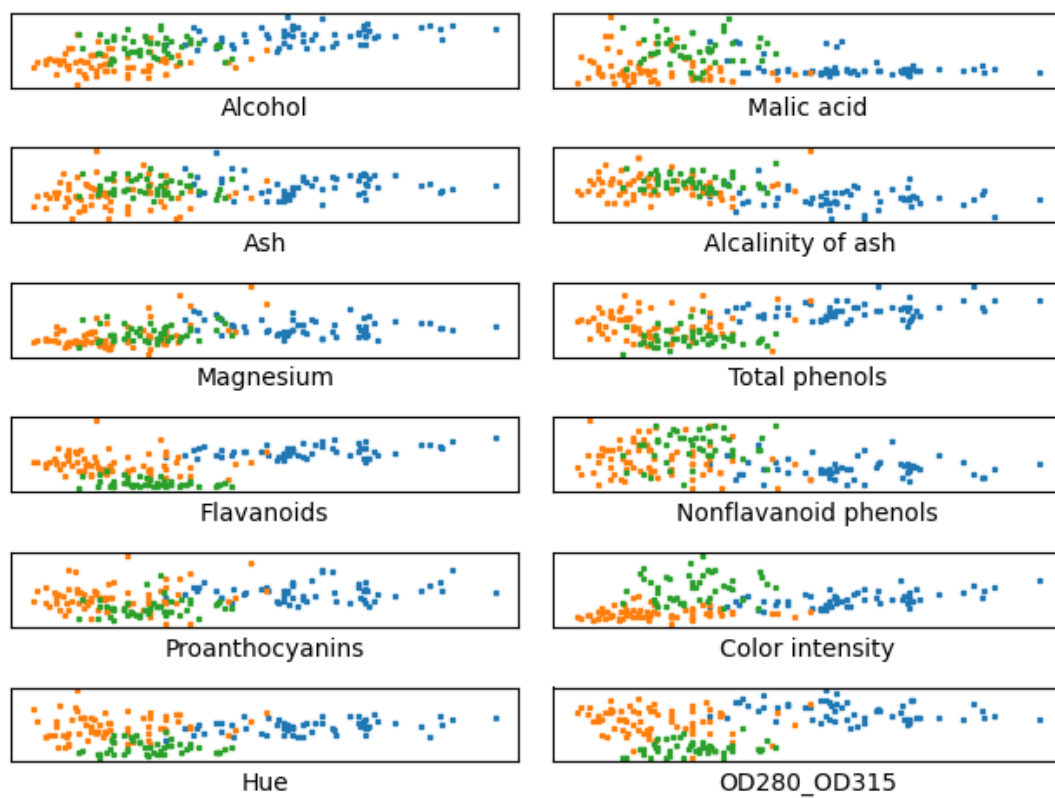
Rysunek 12: Zależność atrybutu 'Nonflavanoid phenols'



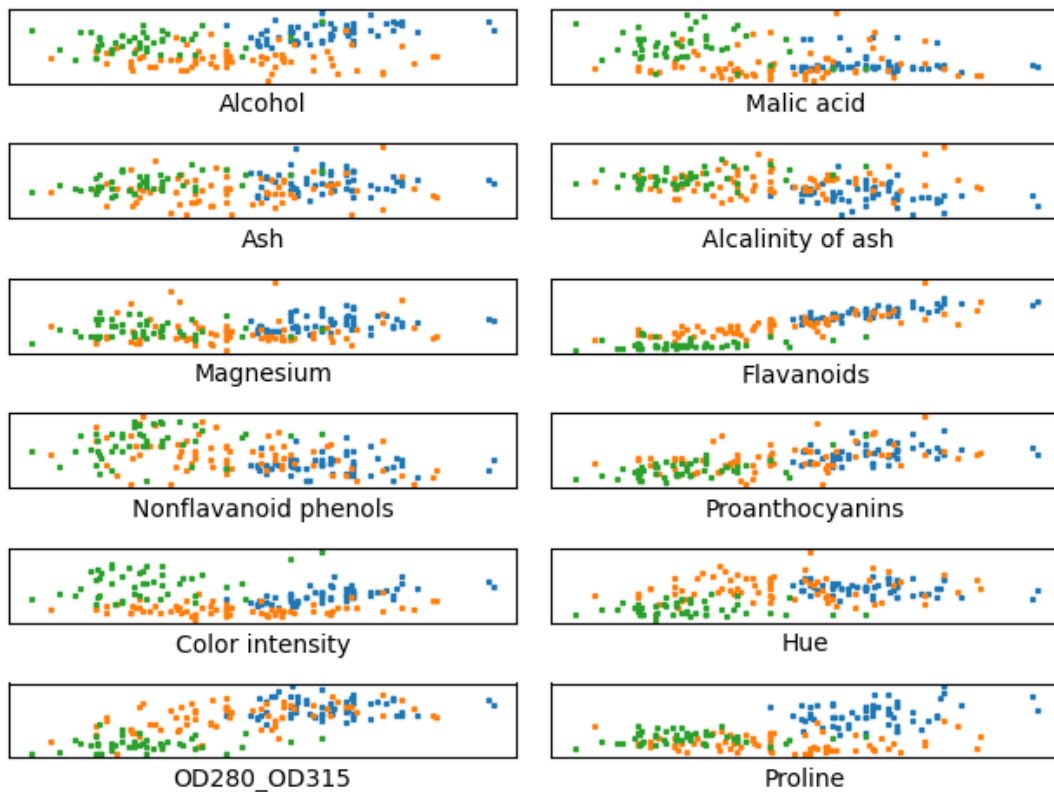
Rysunek 13: Zależność atrybutu ' OD_{280}/OD_{315} '



Rysunek 14: Zależność atrybutu 'Proanthocyanins'



Rysunek 15: Zależność atrybutu 'Proline'



Rysunek 16: Zależność atrybutu 'Total phenols'

Na podstawie powyższych wykresów 4,5,6,...,16 można wywnioskować, że klasyfikacja danych badanego zbioru jest problemem nieseparowalnym liniowo.

1.2.2 Przygotowanie danych do procesu uczenia

Parametrem wyjściowym jest klasa określona liczbą naturalną ze zbioru $\{1, 2, 3\}$, natomiast parametrem wejściowym jest wektor atrybutów opisanych powyżej[1.2]. W celu usprawnienia procesu uczenia parametry wejściowe oraz wyjściowe zostały poddane procesowi normalizacji metodą *min-max*:

$$x' = a + \frac{(x - \min(x)) \cdot (b - a)}{\max(x) - \min(x)}$$

gdzie:

- a - dolna granica nowego przedziału

- b - górna granica nowego przedziału
- x - wektor danych
- x' - znormalizowany wektor danych

Na potrzeby realizacji projektu, przyjęto dla całego zestawu danych przedział normalizacyjny:

$$< -1, 1 >$$

Listing 1 zawiera fragment danych surowych(przed normalizacją), Listing 2 zawiera ten sam fragment danych po procesie normalizacji.

```

1      1,14.23,1.71,2.43,15.6,127,2.8,3.06,.28,2.29,5.64,1.04,3.92,1065
2
3      1,13.2,1.78,2.14,11.2,100,2.65,2.76,.26,1.28,4.38,1.05,3.4,1050
4      1,13.16,2.36,2.67,18.6,101,2.8,3.24,.3,2.81,5.68,1.03,3.17,1185

```

Listing 1: Surowe dane uczące

```

1      -1.00,0.68,-0.62,0.14,-0.48,0.24,0.26,0.15,-0.43,0.19,-0.26,-0.09,0.94,0.12
2
3      -1.00,0.14,-0.59,-0.17,-0.94,-0.35,0.15,0.02,-0.51,-0.45,-0.47,-0.07,0.56,0.10
4      -1.00,0.12,-0.36,0.40,-0.18,-0.33,0.26,0.22,-0.36,0.51,-0.25,-0.11,0.39,0.29

```

Listing 2: Dane uczące po normalizacji

1.3 Zagadnienia teoretyczne

1.3.1 Zasada działania oraz budowa sztucznego neuronu

W celu realizacji projektu z wykorzystaniem zadanego algorytmu uczenia należy określić rodzaj wykorzystanej sieci, a co za tym idzie model sztucznego neuronu.

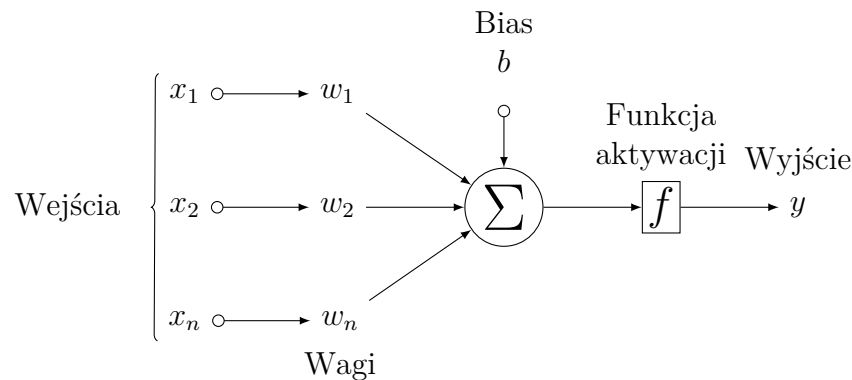
[3]Pierwszy model neuronu zaproponowany przez McColluchoa-Pittsa w 1946 roku, to model neuronu binarnego - jak wskazuje nazwa neuron może na wyjściu zwracać wartość binarną. Tak więc neuron jest jednostką która przetwarza informacje wejściowe, oraz zwraca wynik przetwarzania w postaci wartości wyjściowej.

Wraz z rozwojem dziedziny jaką jest sztuczna inteligencja, struktura sztucznych neuronów uległa zmianie, jednak koncepcja jednostki przetwarzającej dane nie uległa zmianie.

Do budowy sieci neuronowej z wykorzystaniem algorytmu wstecznej propagacji błędów należy wykorzystać sieć typu Feed-Forward[1.3.2], zbudowanej z neuronów typu sigmoidalnego - które strukturą przypominają model McCullocha-Pittsa. Zasada działania neuronu polega na przepływie sygnałów wejściowych poprzez poszczególne fragmenty neuronu do jego wyjścia, bez zastosowania sprzężenia zwrotnego.

W zaprezentowanym modelu(Rys.17) sygnał przechodzi z lewej do prawej strony:

1. wektor danych wejściowych x , oraz powiązany z nim wektor wag w - każda wartość wejściowa x_i jest mnożona przez wagę w_i
2. wyznaczony wektor wejściowy trafia do bloku sumacyjnego - gdzie do sumy wartości wektora dodawana jest wartość przesunięcia $bias$
3. wyjście bloku sumacyjnego jest zarazem wejściem dla argumentu funkcji aktywacji(wartość tą określa się również jako łączne pobudzenie neuronu) f która może przybrać postać funkcji bipolarnej(wyjście w zakresie $< -1, 1 >$) oraz unipolarnej (wyjście w zakresie $< 0, 1 >$)
4. wyjście sygnału z funkcji aktywacji jest równocześnie wyjściem dla neuronu



Rysunek 17: Model neuronu na podstawie [3]

Ogólny wzór wartości wyjścia neuronu:

$$y = f\left(\sum_{i=1}^n (x_i \cdot w_i) + b\right)$$

gdzie:

- y - wyjście neuronu
- x - wektor wejściowy
- b - bias
- w - wektor wag
- f - funkcja aktywacji
- n - ilość wejść

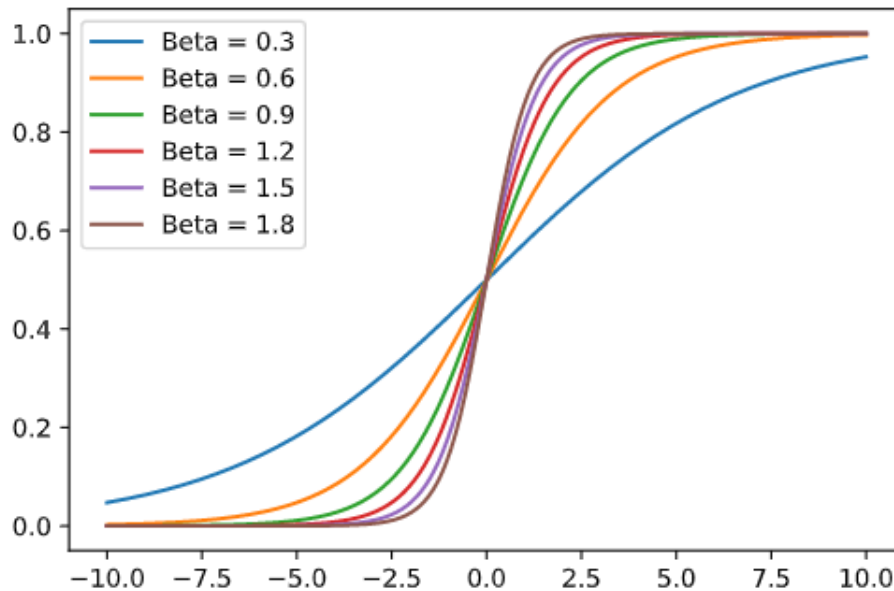
Funkcja unipolarna ma postać[2]:

$$f(x) = \frac{1}{1 + e^{-\beta \cdot x}} \quad (1)$$

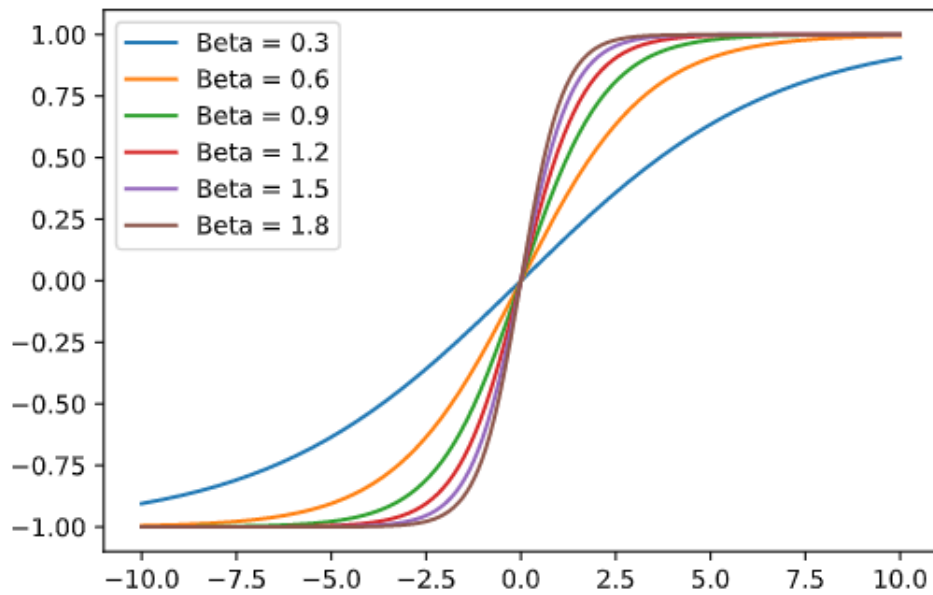
natomiast funkcja bipolarna:

$$f(x) = \frac{2}{1 + e^{-\beta \cdot x}} - 1 \quad (2)$$

w obu przedstawionych definicjach istnieje parametr β , który z reguły określony jest na przedziale $(0, 1 >$. Parametr β wpływa na kształt krzywej wykresu funkcji, co zostało zilustrowane na rysunku 18, oraz 19



Rysunek 18: Wykres funkcji unipolarnej



Rysunek 19: Wykres funkcji bipolarnej

Z przedstawionych powyżej wykresów można wywnioskować, że parametr β wpływa na szerokość przedziału w którym funkcja aktywacji zwróci wartość mieszczącą się w przedziale granicznym funkcji, ale nie będącą wartością graniczną.

W realizacji niektórych sieci neuronowych można spotkać się również z liniową funkcją aktywacji:

$$f(x) = a \cdot x + b$$

1.3.2 Pojęcie warstwy neuronów, oraz sieci neuronowej

Zanim zostanie omówiona budowa oraz działanie sieci neuronowej, należy zdefiniować pojęcie warstwy które stosuje się powszechnie w kontekście sieci neuronowej. Warstwa jest zbiorem neuronów które posiadają kilka wspólnych cech:

- każdy z neuronów w warstwie przyjmuje ten sam wektor wejściowy, natomiast ich wyjścia połączone są w jeden wektor wyjściowy zwany dalej jako wyjście dla danej warstwy
- ilość wejść - z powodu przyjmowania identycznego wektora wejściowego każdy neuron posiada taką samą ilość wejść/wag

Połączenie kilku warstw w strukturę określa się mianem sieci wielowarstwowej, w której można wyróżnić trzy charakterystyczne elementy [4]:

- warstwę wejściową - wejście
- n warstw ukrytych
- warstwę wyjściową

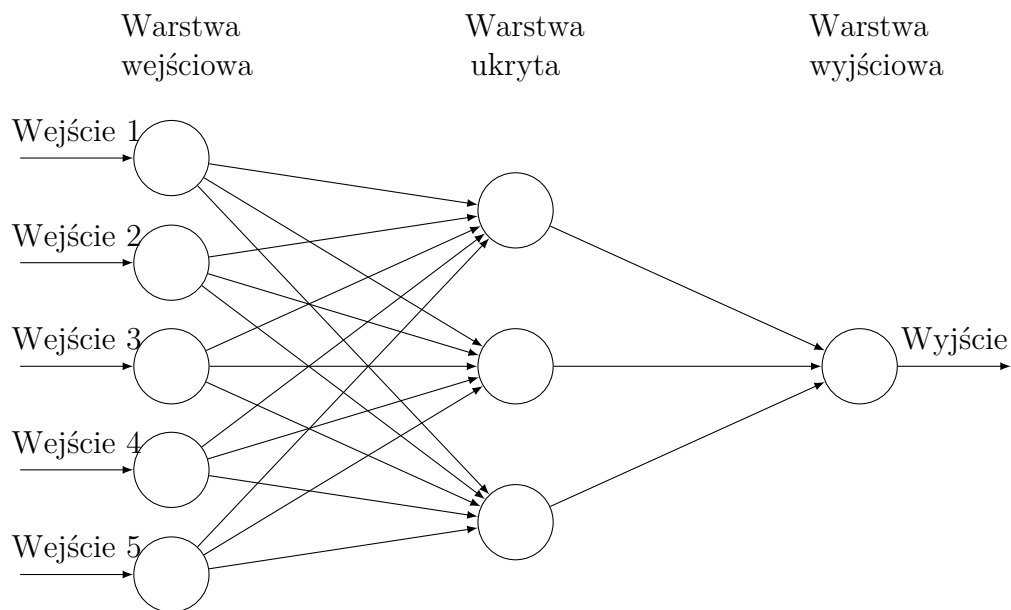
Struktura sieci wielowarstwowej zaprezentowana została na rysunku[20], podobnie jak w przypadku neuronu kierunek przepływu sygnału odbywa się od lewej do prawej strony grafiki.

W prezentowanym przykładzie[20] sieci neuronowej na jej wejście podawany jest wektor wejściowy składający się z 5 wartości, natomiast wektor wyjściowy składa się jedynie z jednej wielkości.

Charakterystyczną cechą dla warstwy wejściowej jest to, że nie bierze ona udziału w procesie uczenia, jej zadaniem jest przekazanie wektora wejściowego sieci do warstw ukrytych, lub bezpośrednio do warstwy wyjściowej (w przypadku modelu sieci jednowarstwowej).

Przebieg sygnału przez sieć jest ściśle powiązany z architekturą sieci. Dla perceptronu (tj. jednokierunkowej sieci typu Feed-Forward) sygnał uzyskany na wyjściu nigdy nie wraca w postaci sprzężenia zwrotnego na wejście sieci/warstwy - propagacja sygnału przebiega od warstwy wejściowej poprzez kolejne warstwy ukryte aż do warstwy wyjściowej z zachowaniem zasad:

- Każda z warstw (oprócz warstwy wejściowej) posiada tyle wejść/wag ile neuronów znajduje się w poprzedniej warstwie
- Wektor wyjściowy warstwy poprzedniej jest w całości przesyłany na wejście każdego neuronu warstwy następnej
- Liczba neuronów warstwy wejściowej musi być identyczna jak długość wektora wejściowego sieci



Rysunek 20: Model sieci wielowarstwowej, z jedną warstwą ukrytą[5]

Ilość warstw w sieci oraz ilość neuronów w warstwie jest ściśle uzależnione od specyfiki problemu przed jakim zostaje postawiona sieć neuronowa. Sieć o określonej architekturze postawiona przed dwoma różnymi problemami może w jednym przypadku w krótkim czasie osiągnąć optymalne wyniki, natomiast w przypadku innego problemu może nigdy nie osiągnąć zadowalających wyników. Dlatego w celu wyznaczenia optymalnych parametrów sieci, przed rozpoczęciem procesu uczenia na dużym zestawie danych, sprawdza się eksperymentalnie konfiguracje parametrów sieci pod względem znalezienia optymalnych ustawień wstępnych.

1.3.3 Proces nauki sztucznej sieci neuronowej

W celu minimalizacji błędu wyjścia sieci względem wartości oczekiwanych na wyjściu, wymagane jest przeprowadzenie procesu nauki sieci, proces ten polega na dopasowaniu parametrów sieci takich jak:

- wagi wejściowe każdego z neuronów w warstwach ukrytych oraz w warstwie wyjściowej
- bias każdego neuronu w warstwach ukrytych oraz w warstwie wyjściowej

Najprostszą metodą nauki jest nauka z nauczycielem (supervised learning), gdzie wykorzystuje się zestaw par danych uczących z czego każda z par składa się z:

- wektor wejściowy - wektor podawany na wejście sieci
- wektor wyjściowy - wektor wartości oczekiwanych na wyjściu sieci, porównywany z faktycznymi wielkościami uzyskanymi na wyjściu sieci

W ogólnym ujęciu proces nauki przebiega w następujący sposób:

1. Na wejście sieci zostaje podany wektor wejściowy
2. Na podstawie wyniku otrzymanego na wyjściu sieci, obliczona zostaje wartość funkcji straty (loss function)
3. Obliczona zostaje wartość funkcji celu (cost function)
4. Na podstawie funkcji celu wyznaczone zostają nowe wartości wag oraz przesunięć - metoda wyznaczania tych wartości zależy od zastosowanego algorytmu uczącego

W przypadku algorytmu największego spadku gradientu funkcja straty jest postaci:

$$e_i = (\hat{y}_i - y_i)$$

gdzie:

- e_i - wartość funkcji straty dla i-tego neuronu warstwy wyjściowej
- $\hat{y}_i$ - wartość uzyskana na wyjściu i-tego neuronu warstwy wyjściowej
- y_i - wartość oczekiwana dla i-tego wyjścia

Funkcja straty wykorzystywana jest do obliczenia wartości funkcji kosztu, która w bezpośredni sposób bierze udział w procesie uczenia, najczęściej stosowane funkcje kosztu[6]:

- **SSE** - Sum Squared Error - sumaryczny błąd kwadratowy

$$E = \frac{1}{2} \sum_{n=1}^m e_n^2 \quad (3)$$

- **MSE** - Mean Squared Error - błąd średniokwadratowy

$$E = \frac{1}{n} \sum_{n=1}^m e_n^2 \quad (4)$$

gdzie:

- n - numer neuronu w warstwie wyjściowej
- m - ilość neuronów w warstwie wyjściowej

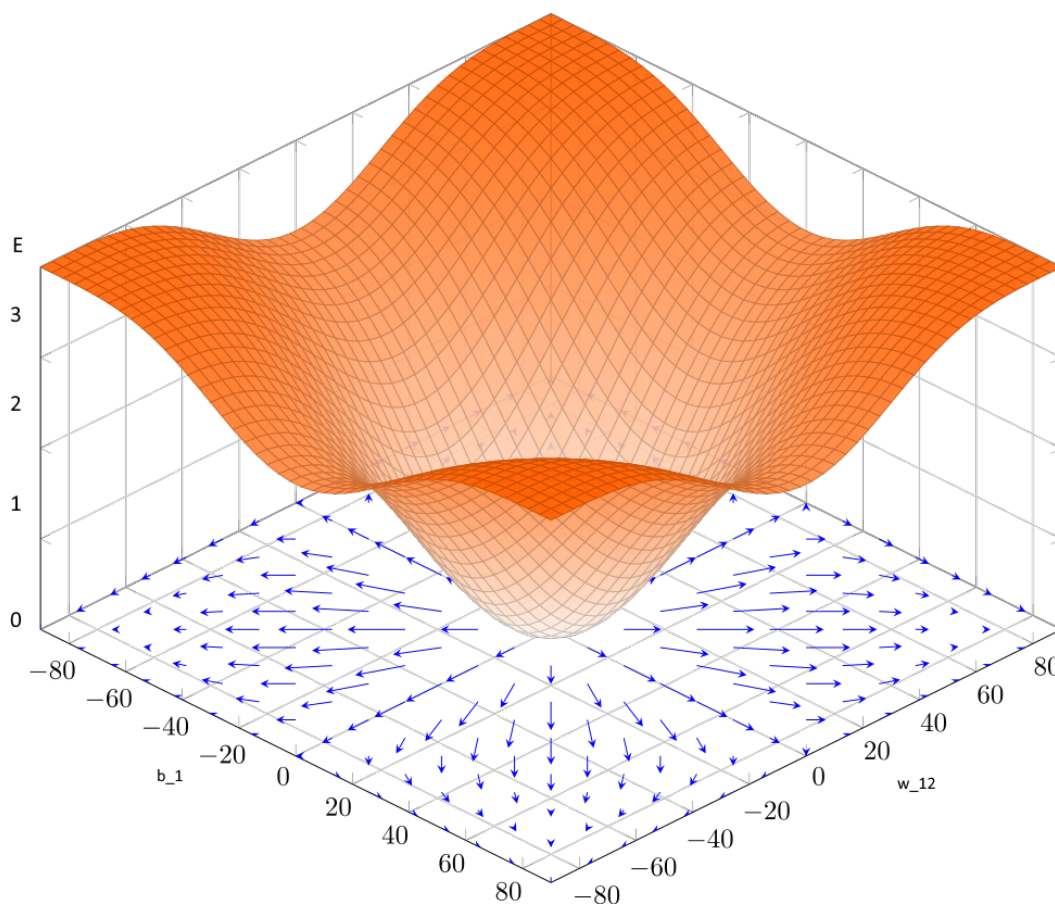
Dla problemu regresji najczęściej stosowaną funkcją jest funkcja błędu średniokwadratowego *MSE*. Funkcja sumarycznego błędu kwadratowego *SSE* ma swoje zastosowanie w problemach dotyczących klasyfikacji.

Funkcje kosztu mówią o tym, o ile sieć neuronowa jest daleko od oczekiwanego wyniku, są pewnego rodzaju wektorem wskazującym algorytmowi uczącemu w którym kierunku powinien przebiegać proces uczenia tj. czy wartości wag powinny zostać zwiększone/zmniejszone oraz o jaką wielkość. Wybranie odpowiedniej funkcji jest kluczowe w celu osiągnięcia optymalnego wyniku uczenia, małym nakładem czasu poświęconego na naukę sieci.

W procesie nauki sieci wyróżnia się trzy metody aktualizacji wag:

- Metoda stochastyczna (online) - parametry sieci aktualizowane są za każdym razem gdy zostanie podana do sieci para ucząca
- Metoda wsadowa (off-line)
 - pełna - parametry neuronów aktualizowane są dopiero po zaprezentowaniu sieci całego zestawu danych uczących
 - mini - stosowana dla dużych zestawów danych uczących. Zbiór danych podzielony jest na podzbiory wielkości kilkuset par uczących, aktualizacja parametrów sieci następuje po zaprezentowaniu całego podzbioru

Pojedynczy cykl w którym zaprezentowano sieci cały zestaw danych uczących zwany dalej będzie epoką.



Rysunek 21: Przykładowy wykres funkcji celu dla pary parametrów sieci [7]

1.3.4 Algorytm wstecznej propagacji błędów

Algorytm wstecznej propagacji błędów, nazywany również algorytmem największego spadku gradientu jest jednym z najprostszych a zarazem skutecznym algorytmem uczenia sieci wielowarstwowych. Działanie algorytmu sprowadza się do wyznaczenia gradientu funkcji celu dla każdego z neuronów sieci. Rysunek 21 przedstawia przykładowy wykres funkcji celu dla pary parametrów neuronu - oś pionowa to wartość funkcji celu, osie poziome to wartości parametrów. Pod wyznaczoną powierzchnią (kolor pomarańczowy) wyznaczono również wektory gradientu wykreślonej funkcji.

Jeżeli powierzchnia jest płaska (na wykresie są to narożniki, oraz punkt centralny) wektory gradientu odpowiadające tym pozycjom są małe co do długości. Warto zauważyć, że wraz ze wzrostem nachylenia powierzchni funkcji, wektory

gradientu stają się dłuższe. Metoda uczenia polega na wykorzystaniu najdłuższego wektora gradientu w punkcie w którym aktualnie znajduje się sieć - wektor gradientu wskazuje na największy wzrost funkcji dlatego w algorytmie stosuje się wektor o odwróconym zwrocie, a więc wektor wskazujący na największy spadek wartości funkcji celu.

Obliczenie wektora gradientu dla każdego neuronu sieci przebiega w inny sposób dla każdego z nich, algorytm polega na wyznaczaniu gradientu zaczynając od wyjściowej warstwy sieci, i podążaniu w kierunku przeciwnym do kierunku przepływu sygnałów w sieci.

Ogólny wzór na gradient dla parametru neuronu określony jest jako[8]:

$$\nabla_{w_{ij}} = \frac{\delta E}{\delta w_{ij}} \quad (5)$$

gdzie:

- δE - pochodna funkcji kosztu
- δw_{ij} - pochodna parametru - w tym przykładzie jest to j-ta waga, i-tego neuronu sieci
- $\nabla_{w_{ij}}$ - szukany gradient

W celu znalezienia pochodnej [5] należy określić bezpośrednie powiązanie pomiędzy funkcją kosztu E a parametrem w_{ij} . W przypadku ostatniej warstwy będzie to:

$$\frac{\delta E}{\delta w_{ij}} = \frac{\delta E}{\delta y_i} \cdot \frac{y_i}{\delta n_i} \cdot \frac{\delta n_i}{\delta w_{ij}} \quad (6)$$

Przedstawiona zależność wynika ze struktury pojedynczego neuronu 1.3.1:

1. Funkcja kosztu zależy w bezpośredni sposób od wyjścia i-tego neuronu y_i

$$\frac{\delta E}{\delta y_i}$$

2. Wyjście neuronu jest bezpośrednio uzależnione od całkowitego pobudzenia neuronu n_i

$$\frac{y_i}{\delta n_i}$$

3. Całkowite pobudzenie neuronu jest bezpośrednio zależne od wartości badanego parametru

$$\frac{\delta n_i}{\delta w_{ij}}$$

Wyznaczając pochodne cząstkowe na podstawie pochodnej funkcji złożonej:

$$\frac{\delta E}{\delta y_i} = \frac{\delta(\frac{1}{2} \cdot \sum_{n=1}^i e_n^2)}{\delta y_i} \quad (7)$$

$$\frac{\delta E}{\delta y_i} = \frac{1}{2} \cdot 2 \cdot (\hat{y}_i - y_i) \cdot (0 - 1) \quad (8)$$

$$\frac{\delta E}{\delta y_i} = -(\hat{y}_i - y_i) = -e_i \quad (9)$$

$$\frac{y_i}{\delta n_i} = f'_i(n_i) \quad (10)$$

$$\frac{\delta n_i}{\delta w_{ij}} = \frac{\delta(w_{i1}a_1 + w_{i2}a_2 + \dots + w_{in}a_n + b_i)}{\delta w_{ij}} \frac{\delta n_i}{\delta w_{ij}} = a_j \quad (11)$$

W przypadku obliczania gradientu dla przesunięcia:

$$\frac{\delta n_i}{\delta b_i} = \frac{\delta(w_{i1}a_1 + w_{i2}a_2 + \dots + w_{in}a_n + b_i)}{\delta b_i} \frac{\delta n_i}{\delta b_i} = 1 \quad (12)$$

Można zauważyć że dla każdego parametru neuronu w istnieje wartość δ_i :

$$\delta_i = e_i \cdot f'_i(n_i) \quad (13)$$

która jest pomocna przy wyznaczaniu gradientu parametrów neuronu:

$$\nabla_{b_i} = \delta_i \quad (14)$$

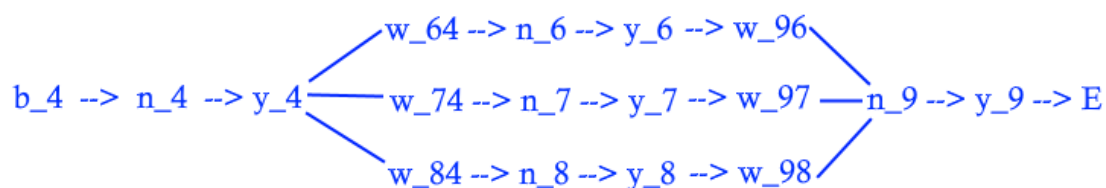
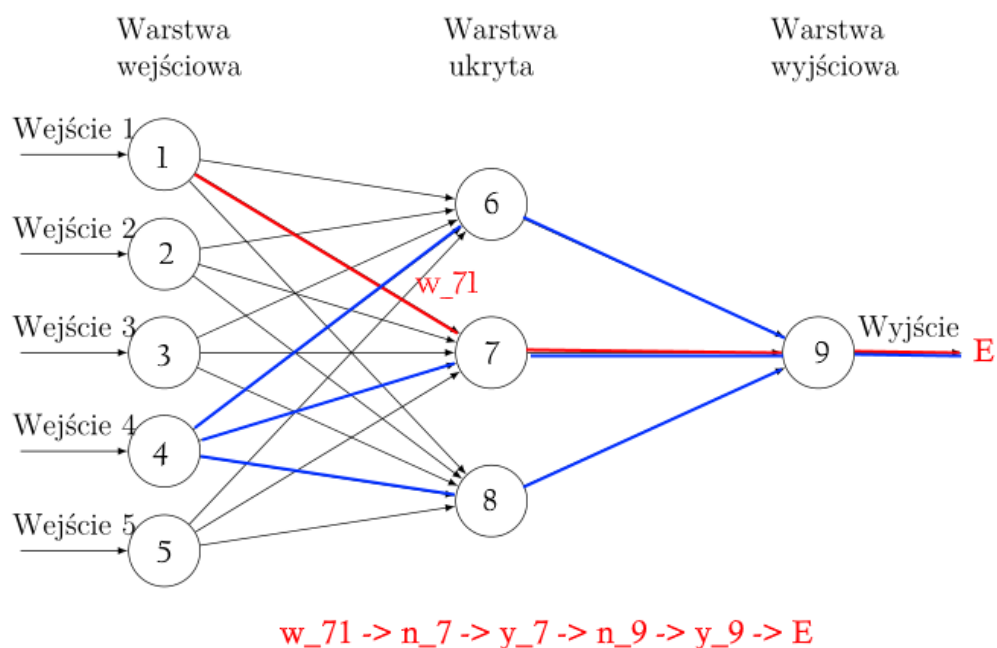
$$\nabla_{w_{ij}} = \delta_i \cdot y_i \quad (15)$$

Algorytm wyznaczania gradientu dla neuronów warstw ukrytych oraz warstwy wyjściowej sprowadza się do następujących czynności:

1. Wyznaczenie wartości: δ_i [13]
2. Przypisanie gradientu przesunięcia: $\nabla_{b_i} = \delta_i$
3. Wyznaczenie gradientu dla każdej wagi neuronu: $\nabla_{w_{ij}} = \delta_i \cdot y_i$

Obliczanie wartości δ_i dla neuronów warstw ukrytych przebiega w podobny sposób tzn. wyznaczany jest ciąg przyczynowo skutkowy który prowadzi z funkcji celu do badanego parametru:

$$\nabla_{w_{ij}} = \frac{\delta E}{\delta w_{ij}}$$



Rysunek 22: Diagram przyczynowo skutkowy, przy wyznaczeniu gradientu dla neuronu 4 oraz 7

Na rysunku 22 zaprezentowano diagram przyczynowo skutkowy który pozwala wyznaczyć wzór na wartość wektora gradientu dla wybranego neuronu.

Algorytm wyznacznia $\nabla_{w_{71}}$ dla drugiego neuronu warstwy ukrytej w sieci[20]:

1. Funkcja kosztu zależy od każdego z wyjść

$$\sum_{i=1}^n \left(\frac{\delta E}{\delta y_i} \right)$$

gdzie n - ilość neuronów warstwy wyjściowej

2. Wyjście każdego neuronu ostatniej warsty zależy od całkowitego pobudzenia neuronu

$$\frac{\delta y_i}{\delta n_i}$$

3. Całkowite pobudzenie każdego neuronu w warstwie zależy od wyjścia badanego neuronu

$$\frac{\delta n_i}{\delta y_7}$$

4. Wyjście badanego neuronu zależy od całkowitego pobudzenia neuronu

$$\frac{\delta y_7}{\delta n_7}$$

5. Całkowite pobudzenie zależy od badanego parametru neuronu

$$\frac{\delta n_7}{\delta w_{71}}$$

Ostatecznie wzór na gradient dla w_{71} :

$$\frac{\delta E}{\delta w_{71}} = \frac{\delta E}{\delta y_9} \cdot \frac{\delta y_9}{\delta n_9} \cdot \frac{\delta n_9}{\delta y_7} \cdot \frac{\delta a_7}{\delta n_7} \cdot \frac{\delta n_7}{\delta w_{71}} \quad (16)$$

Mając wyznaczony gradient dla każdego z neuronów możliwe jest określenie nowych wartości wag oraz przesunięć:

$$b'_i = b_i - \eta \cdot \nabla b_i$$

oraz:

$$w'_{ij} = w_{ij} - \eta \cdot \nabla w_{ij}$$

gdzie η oznacza współczynnik uczenia (Learning rate).

Współczynnik uczenia jest hiperparametrem sieci o wartości z przedziału $(0, 1 >$, wielkość tego współczynnika ma bezpośredni wpływ na proces uczenia się sieci - wykorzystanie współczynnika o dużej wartości będzie skutkować dużymi zmianami wag neuronów, w przypadku małej wartości współczynnika zmiany wag mogą być marginalne. Duża wartość współczynnika uczenia może przyspieszyć proces uczenia, za cenę niższej stabilności procesu (wartość funkcji celu będzie oscylować), oraz ewentualność osiągnięcia jedynie wyniku suboptymalnego. Zbyt mała wartość współczynnika uczenia może w efekcie znacznie wydłużyć czas potrzebny sieci na osiągnięcie optymalnego wyniku, może również doprowadzić do utknięcia sieci w minimum lokalnym, lub do braku zbieżności wyników.

Dobieranie optymalnego współczynnika zależy od specyfiki problemu i przebiega w sposób eksperymentalny.

1.3.5 Adaptacyjny współczynnik uczenia

Opisany wyżej współczynnik uczenia [1.3.4] może być dopasowywany przez algorytm uczący w taki sposób aby proces uczenia był jak najbardziej optymalny. Istnieje kilka algorytmów adaptacji współczynnika uczenia[6]:

- **Bold Driver Method** - jedna z najprostszych metod adaptacji współczynnika uczenia, może być stosowana w każdej metodzie uczenia:

$$\eta_{n+1} = \begin{cases} \eta_n \cdot \gamma^+ & \text{jeżeli } E_n < E_{n-1} \\ \eta_n \cdot \gamma^- & \text{jeżeli } E_n > \beta \cdot E_{n-1} \\ \eta_n & \text{w pozostałych przypadkach} \end{cases}$$

gdzie[9]:

- η_n - współczynnik uczenia w n-tej iteracji
- η_{n+1} - współczynnik uczenia w przyszłej iteracji
- β - współczynnik błędu (Error ratio)

$$1 < \beta$$

- γ^+ - współczynnik przyspieszania uczenia (Learning rate accelerate)

$$\gamma^+ > 1$$

- γ^- - współczynnik spowalniania uczenia (Learning rate decelerate)

$$0 < \gamma^- < 1$$

- E_n - wartość funkcji celu w n-tej epoce
- E_{n-1} - wartość funkcji celu w poprzedniej epoce

- **Search then converge** - metoda sprawdza się w przypadku uczenia wsadowego (Full-batch)

$$\eta_{n+1} = \frac{\eta_0}{1 + \frac{\eta_n}{\tau}}$$

gdzie:

- η_n - współczynnik uczenia w n-tej iteracji
- η_{n+1} - współczynnik uczenia w przyszłej iteracji

- η_0 - stała wybrana na początku procesu uczenia
- τ - ustalona maksymalna ilość epok

Wspomniany wyżej [1.3.5] współczynnik błędu (Error ratio) jest ilorazem:

$$\beta = \frac{E_n}{E_{n-1}}$$

wartość określa maksymalny względny błąd po którym współczynnik uczenia musi zostać zmniejszony.

1.3.6 Metoda momentum

W celu poprawy stabilności procesu uczenia stosuje się metodę zachowania pędu (Momentum). Rozwinięcie algorytmu uczenia o metodę momentum pozwala przesuwać gradient wag nawet po płaskich fragmentach powierzchni funkcji celu, dzięki czemu nie powstają oscylacje mogące wstrzymać proces uczenia w jednym miejscu. Ponadto jeśli współczynnik uczenia jest niski, wprowadzenie pędu do gradientu wag pozwala uniknąć utknięcia w minimum lokalnym.

Aktualizacja wag z wykorzystaniem metody momentum:

$$w_{n+1} = w_n + \Delta w_n + \alpha \Delta w_{n-1} \quad (17)$$

gdzie:

- w_{n+1} - nowa wartość wagi neuronu
- w_n - aktualna wartość wagi
- Δw_n - gradient wagi
- Δw_{n-1} - gradient wagi poprzedniej iteracji algorytmu uczenia
- α - współczynnik momentum

$$0 < \alpha \leq 1$$

2 Implementacja sieci neuronowej

Na potrzeby realizacji sieci neuronowej zaimplementowano bibliotekę w języku C++[10] umożliwiającą w prosty sposób zrealizować dowolną sieć neuronową uczoną metodą wstecznej propagacji błędów, wraz z opcjonalną metodą adaptacyjnego współczynnika uczenia oraz metodą momentum.

Główne cechy biblioteki:

- Dostępne algorytmy inicjalizacji wag:
 - Losowe wartości w zadanym przedziale o rozkładzie normalnym (przedział wartości można dobrać osobno dla wag oraz przesunięć)
 - Inicjalizacja algorytmem Nguyen-Widrow
 - Inicjalizacja wartościami losowymi o ustalonej entropii
- Dostępne funkcje aktywacji:
 - funkcja unipolarna
 - funkcja bipolarna
 - funkcja liniowa
- Możliwość zmiany metody aktualizacji wag, dostępna jest metoda wsadowa, stochastyczna oraz mini-batch
- Dla metody momentum przewidziano możliwość rozszerzenia wektora pędu o dodatkowe wartości
- Możliwość indywidualnego dopasowania hiperparametrów dla pojedynczego neuronu/warstwy lub całej sieci
- Implementacja dwóch metod adaptacji współczynnika uczenia: *Bold Driver* oraz *Search then converge*

Kod biblioteki został zorganizowany tak, że poszczególne elementy sieci neuronowej są zorganizowane w struktury zawierające informacje o hiperparametrach.

Głównym elementem biblioteki jest klasa *Net*[1] która zawiera informacje o parametrach uczenia sieci, wektor zawierający informacje o poszczególnych warstwach, metody pozwalające na odczytanie danych uczących, oraz metody umożliwiające archiwizację parametrów uczenia w formie pliku w formacie *json*.

```
114 * @arg max_epoch - maximum number of training epochs
115 * @arg error_goal - when SSE is less or equal error goal, stops training
116 * @arg return - LearningOutput
```

```

117  */
118  LearnOutput train(double max_epoch = 1000, double error_goal = 0);
119
120  /**
121   * Feed network with sample
122   * just gets output value of each neuron
123   * @arg sample_number - index of sample in input_data vector
124   */
125  void feed(uint32_t sample_number);
126
127  /**
128   * Calculate delta for each neuron
129   * @arg sample_number - index of sample in input_data vector
130   */
131  void get_delta(uint32_t sample_number);
132
133  /**
134   * Calculates error
135   * @arg sample_number - number of sample in input_data vector
136   */
137  double get_loose(uint32_t sample_number);
138
139  /**
140   * Updates weights for all neurons in net
141   */
142  void update_weights();
143
144  /**
145   * Calculate gradient
146   * @arg sample_number - number of sample in input_data vector
147   */
148  void learn(uint32_t sample_number);
149
150  /**
151   * Setup internal vectors according to net properties
152   * @arg hidden_layers - vector of sizes of hidden layers
153   * @arg params - blueprint for each neuron
154   */
155  void setup(std::vector<uint32_t> hidden_layers, LearnParams params);
156

```

```

157  /**
158   * Constructor of neural network
159   * @arg input_data - input data set
160   * Given data set will be divided to subsets: training, validation and test set
161   * Each call of train makes new sets
162   * @arg subsets_ratio - Percentage representation of training, validation, and t
163   */
164  Net(pattern_set &input_data, std::array<double, 2> subsets_ratio);
165

```

Listing 1: Fragment pliku nagłówkowego Net.h

Konstruktor klasy jako argument przyjmuje strukturę zawierającą wektor par uczących, oraz dwuelementową tablicę *subsets_ratio*. W konstruktorze obliczony zostaje rozmiar zbioru testowego oraz zbioru treningowego(uczącego), na podstawie wartości przekazanych w tablicy *subsets_ratio* - jej pierwszy element wskazuje procentowy udział zbioru treningowego w całym przekazanym zbiorze danych, natomiast druga wartość to procentowy udział zbioru testowego w całym zbiorze danych uczących.

Przed rozpoczęciem procesu nauki należy wykonać na obiekcie sieci metodę *setup*, do której muszą zostać przekazane argumenty:

- *hidden_layers* - wektor liczb całkowitych, wskazuje rozmiar każdej z warstw ukrytych
- *params* - struktura określająca parametry uczenia [2]

Sieć domyślnie inicjalizuje warstwę wejściową oraz wyjściową tak aby ich rozmiar zgadzał się z rozmiarem wektorów danych uczących.

```

159  struct LearnParams
160  {
161      /**
162           * Batch size
163           */
164      uint32_t batch_size;
165
166      /**
167           * Initial learning rate
168           */
169      double learning_rate;
170
171      /**

```

```

172         * Error ratio
173         */
174     double error_ratio;
175
176     /**
177         * Aka. gamma+
178         * Note: to use adaptive learning both values must be set
179         */
180     double learning_accelerating_consts;
181
182     /**
183         * Aka. gamma-
184         * Note: to use adaptive learning both values must be set
185         */
186     double learning_decelerating_consts;
187
188     /**
189         * Momentum consts
190         */
191     double momentum_consts;
192
193     /**
194         * Weights deltas vector size for momentum method
195         */
196     uint32_t momentum_delta_vsize;
197
198     /**
199         * Momentum rate
200         */
201     double momentum_rate;
202
203     /**
204         * Beta parameter used in activation function
205         */
206     double beta_param;
207
208     /**
209         * Initial range for weights
210         */
211     rand_range weights_range;

```

```

212
213     /**
214         * Initial range for bias
215     */
216     rand_range bias_range;
217
218     /**
219     * Pointer to activation function
220     */
221     func_ptr activation;
222
223     /**
224     * Pointer to derivative of activation function
225     */
226     func_ptr derivative;
227
228     /**
229     * Pointer to initialization funtion
230     */
231     init_func_ptr init_function;
232
233     /**
234         * Default constructor
235         * sets all values to default
236     */
237     LearnParams()
238     {
239         //Default learn method is batch
240         this->batch_size = 0;
241
242         this->learning_rate = 0.01;
243
244         this->learning_accelerating_constans = 1.05;
245         this->learning_decelerating_constans = 0.7;
246
247         this->error_ratio = 1.04;
248
249         this->momentum_constans = 0.9;
250
251         //Size of vector for weights deltas for each neuron

```

```

252     //Vector of deltas is necessary for momentum method
253     this->momentum_delta_vsize = 1;
254
255     this->beta_param = 1.0;
256

```

Listing 2: Definicja struktury LearnParams

Zmienna *batch_size* określa rozmiar pojedynczego cyklu uczenia sieci, domyślnie jest to wartość równa rozmiarowi zbioru treningowego, tak więc domyślnie sieć korzysta z metody wsadowej.

```

283 void Net::get_delta(uint32_t sample_number)
284 {
285     //Find delta for each neuron in each layer
286
287     //Calculate delta of the output layer
288     auto &last_layer = this->layers.back();
289     for (uint32_t neuron_it{0}; neuron_it < last_layer.neurons.size(); neuron_it++)
290     {
291         auto &neuron = last_layer.neurons[neuron_it];
292         neuron.delta = (this->input_data[sample_number].output[neuron_it] - neuron
293     }
294
295     //Hidden layers
296     for (uint32_t layer_it{static_cast<uint32_t>(this->layers.size()) - 2}; layer_
297     {
298         auto &layer = this->layers[layer_it];
299         for (uint32_t neuron_it{0}; neuron_it < layer.neurons.size(); neuron_it++)
300         {
301             auto &neuron = layer.neurons[neuron_it];
302
303             neuron.delta = 0;
304             for (uint32_t next{0}; next < this->layers[layer_it + 1].neurons.size(
305             {
306                 auto &next_neuron = this->layers[layer_it + 1].neurons[next];
307                 neuron.delta += next_neuron.weights[neuron_it] * next_neuron.delta
308             }
309
310             //calculate the delta for current neuron
311             neuron.delta *= neuron.derivative_output;
312         }

```



```

313     }
314 }

```

Listing 3: Definicja funkcji obliczającej wartości δ_i dla neuronów

Listing 3 zawiera iteracyjną wersję algorytmu wstecznej propagacji - wartości delta obliczane są rozpoczynając od warstwy wyjściowej, w kierunku warstwy wejściowej.

```

330 void Net::update_weights()
331 {
332     //Adaptive learning rate:
333     if (this->epoch && this->learn_parameters.learning_accelerating_consts > 0 &&
334         {
335         double error_r = this->SSE / this->prev_SSE;
336         if (error_r > this->learn_parameters.error_ratio)
337             this->update_learning_rate(LearningRateUpdate::Decrease);
338         else if (error_r < 1)
339             this->update_learning_rate(LearningRateUpdate::Increase);
340     }
341
342     //Accumulate weight and bias updates form each run
343     for (uint32_t i{1}; i < this->layers.size(); i++)
344     {
345         for (auto &neuron : this->layers[i].neurons)
346         {
347             neuron.bias_update = std::accumulate(VEC_RANGE(neuron.batch.bias_delta),
348             neuron.bias_update, 0.0);
349             for (uint32_t it{0}; it < neuron.weights.size(); it++)
350                 neuron.weight_update[it] = std::accumulate(VEC_RANGE(neuron.batch.w_delta[it]),
351                 neuron.weight_update[it], 0.0);
352         }
353     }
354
355     //Momentum:
356     if (learn_parameters.momentum_delta_vsize && learn_parameters.momentum_consts > 0 &&
357         {
358         uint32_t index = (this->epoch + 1) % this->learn_parameters.momentum_delta_vsize;
359         for (uint32_t layer_it{1}; layer_it < this->layers.size(); layer_it++)
360         {
361             for (auto &neuron : this->layers[layer_it].neurons)
362             {
363                 neuron.bias_update = (neuron.bias_update * (1.0 - neuron.learn_parameters.momentum_consts) +
364                 neuron.batch.bias_delta[index]);
365                 for (uint32_t w_it{0}; w_it < neuron.weights.size(); w_it++)
366                     neuron.weight_update[w_it] = (neuron.weight_update[w_it] * (1.0 - neuron.learn_parameters.momentum_consts) +
367                     neuron.batch.w_delta[w_it][index]);
368             }
369         }
370     }
371 }

```

```

363         neuron.bias_deltas[index] = neuron.bias_update;
364
365         for (uint32_t it{0}; it < neuron.weights.size(); it++)
366         {
367             neuron.weight_update[it] = (neuron.weight_update[it] * (1.0 - 1)
368             neuron.weights_deltas[it][index] = neuron.weight_update[it];
369         }
370     }
371 }
372 }
373
374 //Finally update weights and biases
375 for (uint32_t i{1}; i < this->layers.size(); i++)
376 {
377     for (auto &neuron : this->layers[i].neurons)
378     {
379         neuron.bias -= neuron.bias_update;
380
381         for (uint32_t it{0}; it < neuron.weights.size(); it++)
382             neuron.weights[it] -= neuron.weight_update[it];
383     }
384 }
385 }

```

Listing 4: Definicja funkcji aktualizującej wagi

Listing 4 zawiera definicję funkcji odpowiedzialnej za aktualizację wag oraz przesunięć, oraz posiada zaimplementowane metody adaptacji współczynnika uczenia wraz z metodą momentum.

```

185 for (; this->epoch < max_epoch; this->epoch++)
186 {
187     this->batch_it = 1;
188     epoch_SSE = 0;
189
190     //Make full run over training data
191     for (uint32_t it{0}; it < train_set.size(); it++)
192     {
193         index = train_set[it];
194         this->learn(index);
195         classification_train[it] = this->get_classification_succes(index);
196     }

```

```

197     //if batch ends - update weights and biases
198     if ((this->batch_it % this->learn_parameters.batch_size) == 0)
199     {
200         this->SSE *= 0.5;
201         this->prev_SSE = this->SSE;
202         epoch_SSE += this->SSE;
203         this->update_weights();
204         this->batch_it = 1;
205         this->SSE = 0.0;
206     }
207     else
208         batch_it++;
209
210 }
211
212 //Stuff for later analyse
213 out.train_set_SSE.push_back(epoch_SSE);
214 out.train_set_MSE.push_back(epoch_SSE / static_cast<double>(train_set.size()));
215
216 classification_accuracy = 0;
217 for (auto value : classification_train)
218     classification_accuracy += static_cast<double>(value);
219 out.train_set_accuracy.push_back(classification_accuracy / static_cast<double>
220
221 epoch_SSE = 0;
222 for (uint32_t it{0}; it < test_set.size(); it++)
223 {
224     index = test_set[it];
225     this->feed(index);
226     epoch_SSE += this->get_loose(index);
227     classification_test[it] = this->get_classification_succes(index);
228 }
229 out.test_set_SSE.push_back(epoch_SSE);
230 out.test_set_MSE.push_back(epoch_SSE / static_cast<double>(train_set.size()));
231
232 classification_accuracy = 0;
233 for (auto value : classification_test)
234     classification_accuracy += static_cast<double>(value);
235 out.test_set_accuracy.push_back(classification_accuracy / static_cast<double>(
236

```

```

237     if (out.train_set_SSE.back() < error_goal)
238         break;
239 }

```

Listing 5: Definicja funkcji realizującej uczenie

Listing 5 zawiera najważniejszy dla całej biblioteki fragment funkcji *train* odpowiedzialnej za realizację procesu uczenia. Na początku każdej epoki[1.3.3] parametry tymczasowe sieci są przywracane do wartości domyślnych, w następnym kroku program wykonuje proces uczenia na zbiorze uczącym. Zestaw danych uczących jest dla każdego wywołania funkcji *train* dobierany w sposób losowy, z zachowaniem proporcji dla każdej z klas ze zbioru uczącego. Po ukończeniu procesu nauki wyniki dla każdej epoki zapisywane są do struktury *LearnOutput* - która zawiera wszystkie parametry przed oraz po wykonaniu nauki, wraz z wartościami pośrednimi, dla każdego neuronu z osobna. Przed zakończeniem epoki następuje proces sprawdzania efektów nauki na zbiorze testowym - dane z tego zbioru nie biorą udziału w nauce a jedynie są prezentowane sieci w celu sprawdzenia poprawności klasyfikacji. Warto zaznaczyć że podzbiory *train_set* oraz *test_set* są rozłączne.

3 Eksperymenty

3.1 Eksperyment I - Badanie wpływu hiperparametrów na szybkość uczenia sieci

Celem eksperymentu jest znalezienie konfiguracji hiperparametrów, dla której sieć osiągnie maksymalną poprawność klasyfikacji, w jak najkrótszym czasie. Listing 6 zawiera kod realizujący pierwszy etap eksperymentu - badanie wpływu architektury sieci na szybkość uczenia, przyjęto następujące wartości domyślne:

S1 (neurony warstwy I)	<1,25>
S2 (neurony warstwy II)	<1,25>
Learning rate	0.01
Learning rate accelerating	1.05
Learning rate decelerating	0.7
Error ratio	1.04
Momentum constans	0.9
Funkcja aktywacji	Bipolarna
Stosunek wielkości zbiorów	80% : 20%
Funkcja inicjalizacji wag	Losowa z ustaloną entropią
Zakres inicjalizacji wag	<-0.1,0.1>
Zakres inicjalizacji biasów	<-0.01,0.01>
Metoda uczenia	Wsadowa
Maksymalna ilość epok	1000
Oczekiwana wartość funkcji celu	$SSE < 0.25$

Tabela 2: Zestawienie hiperparametrów sieci wraz z parametrami uczenia

```

1  #include <lib/net.h>
2
3  int main()
4  {
5      pattern_set input;
6      Net::read_file("INPUT_DATA.txt", input);
7      Net::open_file("OUTPUT.json");
8      auto net = Net(input, {0.8,0.2});
9
10     LearnParams params = LearnParams();
11     params.init_function = InitFunction::const_rand;
12     params.bias_range = rand_range(-0.01,0.01);
13     params.weights_range = rand_range(-0.1,0.1);
14     LearnOutput output;
15
16     uint32_t S1_MIN{1};
17     uint32_t S1_MAX{25};
18
19     uint32_t S2_MIN{1};
20     uint32_t S2_MAX{25};
21

```

```

22     uint32_t left{(S1_MAX - S1_MIN + 1) * (S2_MAX - S2_MIN + 1)};
23     uint32_t i{1};
24
25     for(uint32_t S1{S1_MIN}; S1<=S1_MAX; S1++){
26         for(uint32_t S2{S2_MIN}; S2<=S2_MAX; S2++){
27             net.setup({S1, S2}, params);
28             output = net.train(1000, 0.25);
29             Net::save_output("OUTPUT.json", output);
30             std::cout << "Iterations left: " << left - i++ << "   S1:" << S1 << "
31         }
32     }
33
34     Net::close_file("OUTPUT.json");
35     return 0;
36 }

```

Listing 6: Kod programu realizujący pierwszy etap eksperymentu

W celu zminimalizowania wpływu wartości początkowych wag, wybrano minimalny zakres wartości, oraz zastosowano funkcję losującą o ustalonej entropii. Ponadto wyłączono losowy dobór wartości do podzbiorów [2].

Przedstawiony problem jest problemem klasyfikacji, dlatego za funkcję celu wybrano sumaryczny błąd kwadratowy (SSE). Wartość oczekiwana funkcji celu jest zależna od specyfiki danych, w wykorzystywanym zbiorze danych istnieją trzy klasy, gdzie identyfikatory klas należą do zbioru $\{-1, 0, 1\}$. Na potrzeby sklasyfikowania wyników sieci, obrano przedziały które w równych odległościach oddzielają sąsiednie klasy:

1. Wartości w przedziale $< -1.0, -0.5 >$ - I klasa
2. Wartości w przedziale $(-0.5, 0.5)$ - II klasa
3. Wartości w przedziale $< 0.5, 1.0 >$ - III klasa

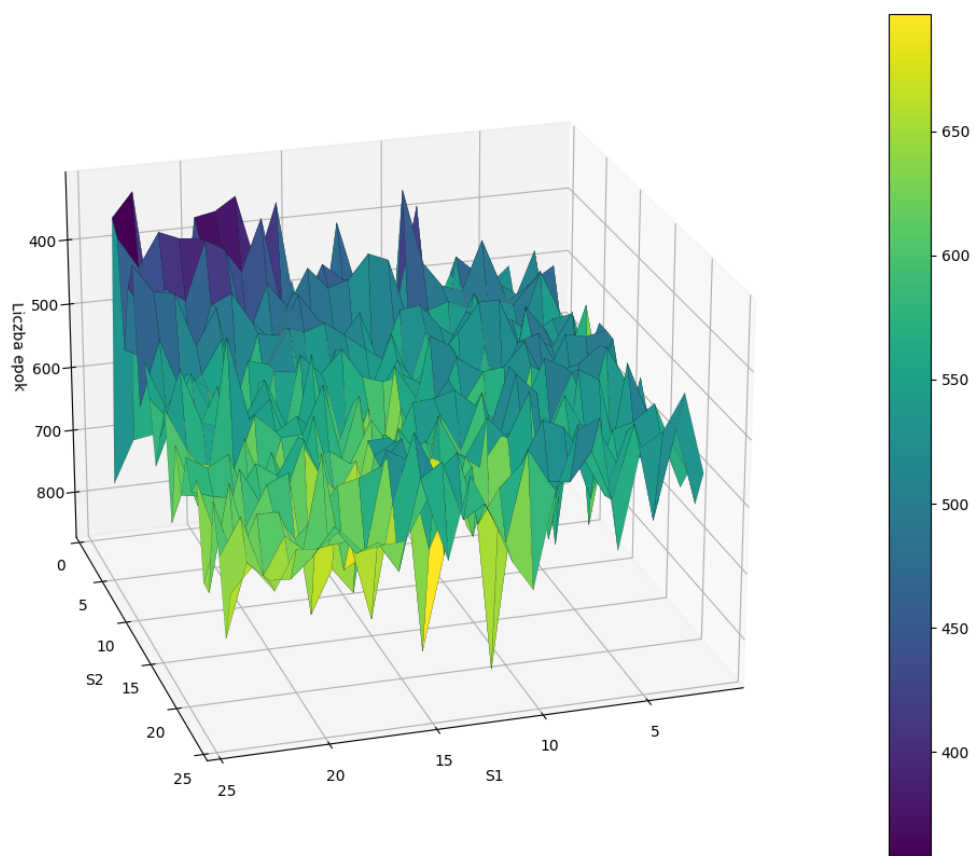
Podane przedziały klasyfikacji uwzględniają specyfikę funkcji aktywacji - wartości funkcji nigdy nie przekroczą zakresu $< -1.0, 1.0 >$ dlatego skraje przedziały zostały skorygowane.

Aby sieć osiągnęła maksimum poprawności w klasyfikacji, sumaryczny błąd kwadratowy musi być mniejszy niż kwadrat połowy szerokości przedziału klasyfikacji - kwadrat wynika ze specyfiki funkcji celu. Dla wykorzystywanego zbioru danych szerokość przedziału dla jednej klasy wynosi 1.0, więc oczekiwana wartość funkcji celu musi być większa niż 0.25, aby sieć klasyfikowała podane wartości bezbłędnie.

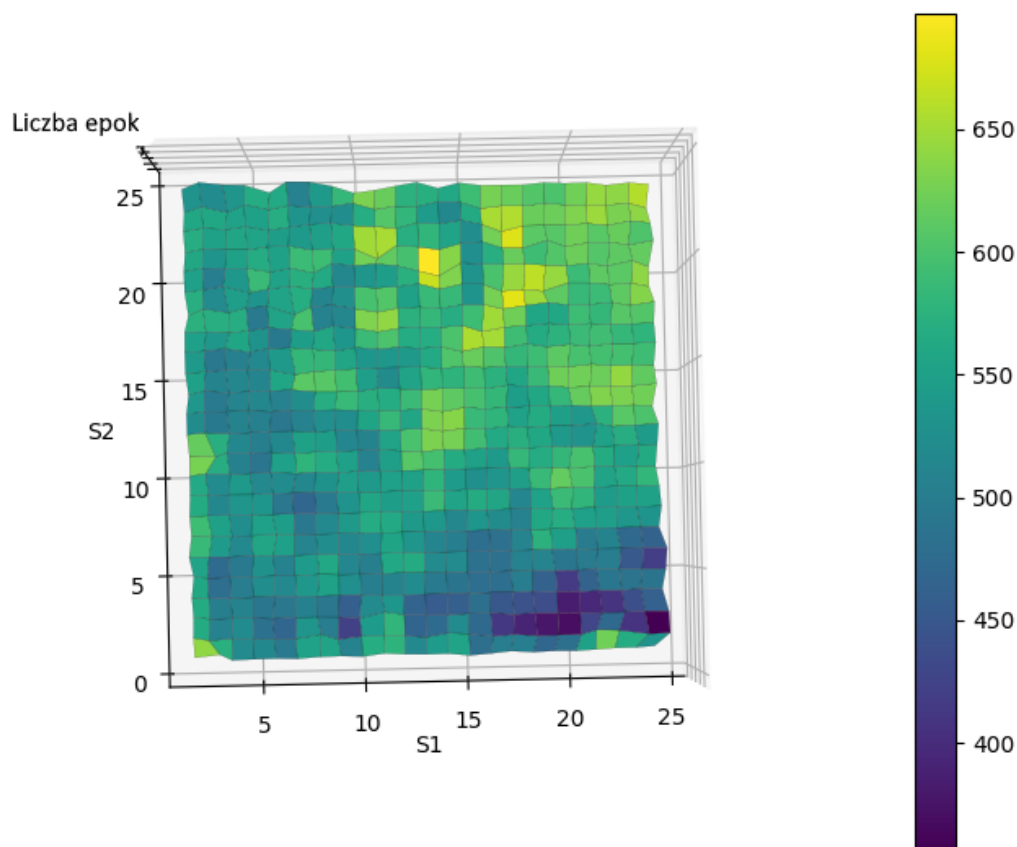
Sposób wyznaczania oczekiwanej wartości funkcji celu: jeżeli wartość funkcji celu pod koniec epoki wynosi < 0.25 , to oznacza to tyle, że suma odległości pomiędzy wartościami uzyskanymi na wyjściu sieci a wartościami oczekiwanymi jest mniejsza od maksymalnej odległości dla pojedynczej próby której wynik klasyfikacji jest poprawny. Niemożliwe jest więc błędne sklasyfikowanie którejkolwiek z próbek. Wykresy [25],[26] wskazują na powiązanie funkcji celu wraz ze skutecznością sieci w klasyfikowaniu, mimo że sieć osiąga $SSE > 0.25$, to skuteczność w klasyfikacji jest już na poziomie 100%, nie oznacza to jednak, że za każdym razem gdy sieci zostaną zaprezentowane nowe dane, to sieć nie popełni błędu.

3.1.1 Badanie wpływu architektury sieci (hiperparametry S1, S2)

Wyniki poddano analizie, która polegała na zestawieniu badanych wielkości S1 oraz S2. Dla każdej z kombinacji badanych hiperparametrów proces uczenia zakończył się przed osiągnięciem maksymalnej ilości epok, co świadczy o tym, że przedstawiony zbiór danych jest niewymagający. W celu poprawnej analizy danych wykreślono powierzchnię funkcji minimalnej ilości epok potrzebnych do osiągnięcia $SSE < 0.25$ [23][24] której argumentami są hiperparametry S1,S2.

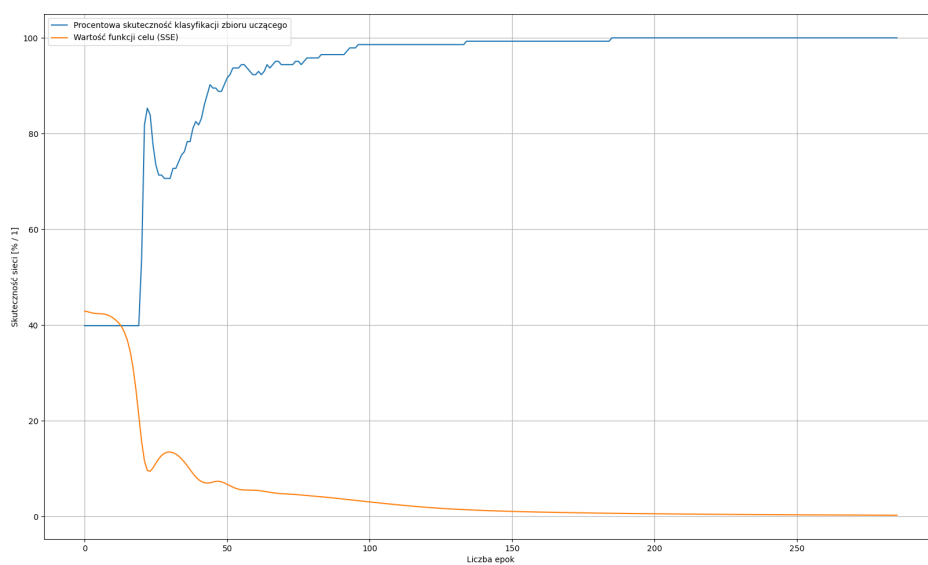


Rysunek 23: Wykres powierzchniowy, ilość epok potrzebnych do nauczenia sieci

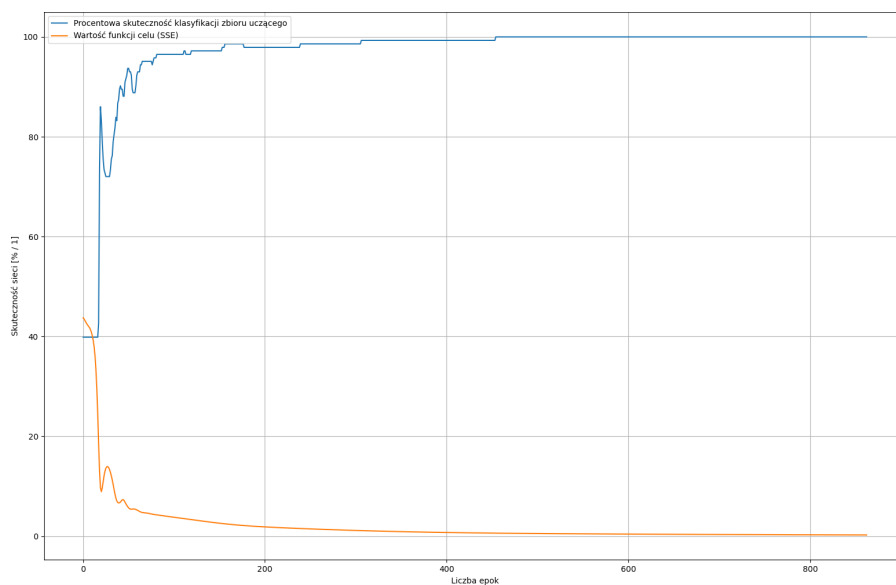


Rysunek 24: Wykres powierzchniowy, ilość epok potrzebnych do nauczania sieci

Najwydajniejszą kombinacją hiperparametrów $S1, S2$ jest para $(25, 2)$ ^[25], dla której osiągnięcie zadanej sprawności zajęło 285 cykli (epok). Natomiast najgorszą kombinacją jest kombinacja $(11, 21)$ ^[26] - co potwierdza słuszność założenia, że ilość neuronów w kolejnych warstwach ukrytych powinna się zmniejszać.



Rysunek 25: Wykres funkcji celu, oraz procentowej skuteczności w klasyfikacji w kolejnych epokach - $S1 = 25$, $S2 = 2$



Rysunek 26: Wykres funkcji celu, oraz procentowej skuteczności w klasyfikacji w kolejnych epokach - $S1 = 11$, $S2 = 21$

Analizując rysunek 24 można wywnioskować, że najwydajniejsze kombinacje hiperparametrów S1, oraz S2 znajdują się w przedziałach:

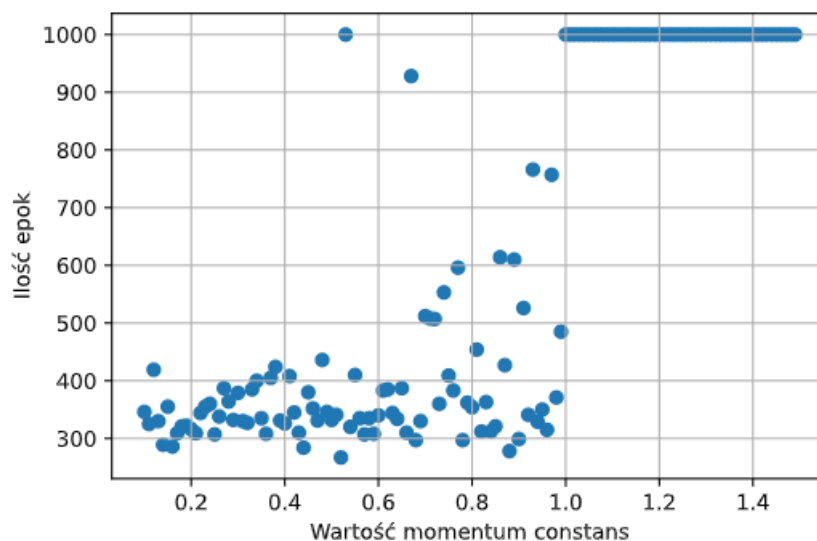
- dla S1 - $< 15, 25 >$
- dla S2 - $< 1, 8 >$

dlatego dalsza analiza kolejnych hiperparametrów będzie odbywać się na sieciach o architekturze zdefiniowanej przez powyższe wartości. wraz ze wzrostem ilości neuronów na poszczególnych warstwach, a ponadto większa ilość neuronów w warstwie skutkuje większym zużyciem zasobów jednostki obliczeniowej oraz czasu, potrzebnego do osiągnięcia zadanego celu.

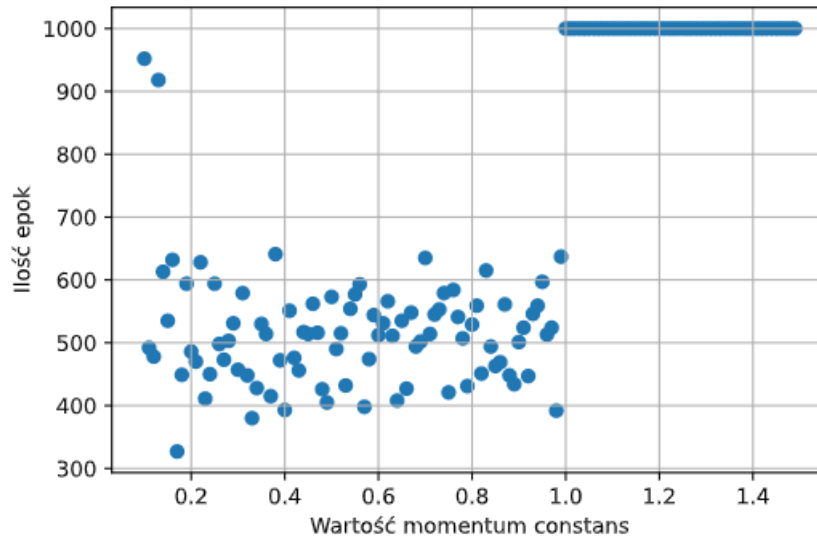
3.1.2 Badanie wpływu momentum constans

Badanie wpływu wielkości współczynnika momentum constans na szybkość uczenia odbywać się będzie dla parametrów określonych wcześniej[2]. W ramach eksperymentu zmianie ulegać będzie wartość współczynnika momentum constans, jako zakres zmian przyjęto przedział $< 0.1, 1.5 >$ ze skokiem 0.01

Wyniki eksperymentu przy dwóch różnych architekturach sieci zaprezentowano na rysunkach [27], [28]



Rysunek 27: Wykres zależności szybkości uczenia od współczynnika momentum constans S1 = 25, S2 = 2

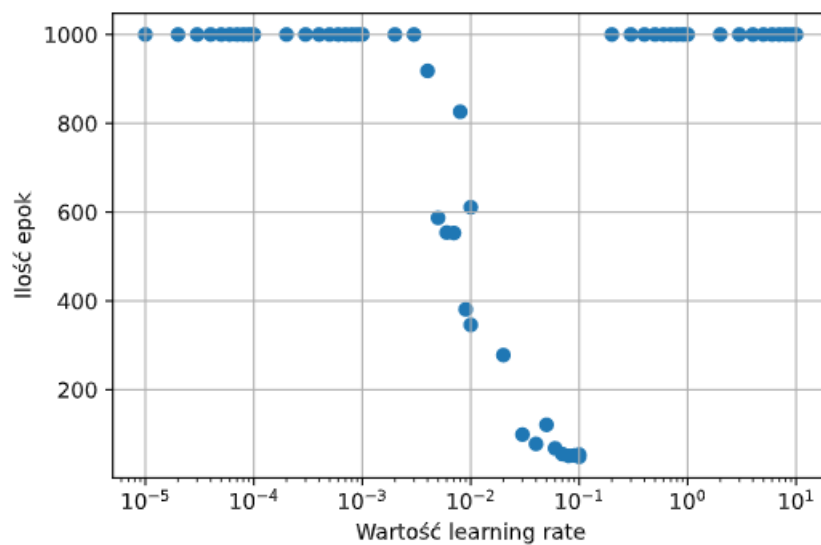


Rysunek 28: Wykres zależności szybkości uczenia od współczynnika momentum constans $S1 = 15$, $S2 = 6$

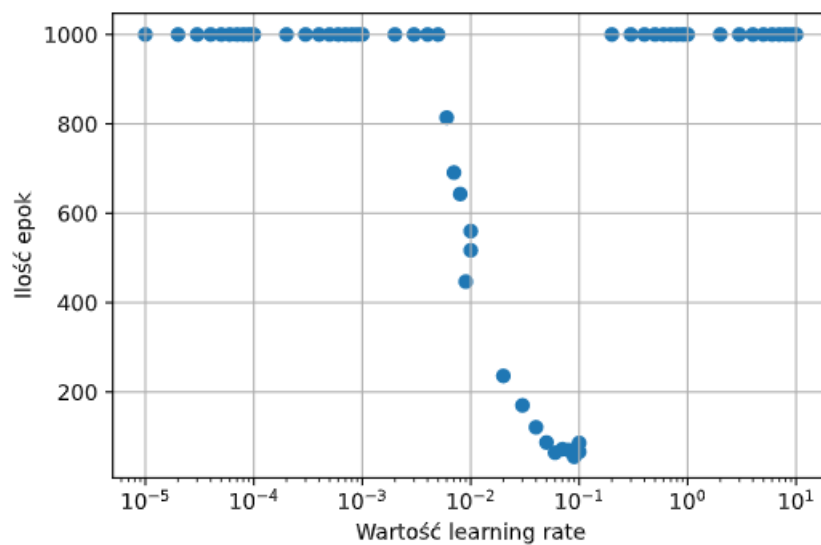
Otrzymane wyniki wykazują że wprowadzanie wartości momentum constans powyżej 1.0 jest bezcelowe. Dla najbardziej optymalnej architektury sieci ($S1 = 25$, $S2 = 2$) szybkość uczenia jest na poziomie niższym niż 400 epok, niezależnie od wartości momentum constans (z małymi wyjątkami). Najlepszy wynik dla omawianej sieci to 258 epok, przy momentum constans 0.58. Dla mniej optymalnej architektury ($S1 = 15$, $S2 = 6$) szybkość uczenia sieci jest mniejsza, ale występuje mniej dużych odchyleń od średniej, w tym przypadku najlepszy rezultat - 328 epok uzyskano dla badanego parametru równego 0.16.

3.1.3 Badanie wpływu learning rate

Podobnie jak we wcześniejszym eksperymencie do wykonania badań wykorzystane zostaną dwie sieci neuronowe o architekturze ($S1 = 25$, $S2 = 2$) [29] oraz ($S1 = 15$, $S2 = 6$) [30], parametry sieci zdefiniowane zostały w tabeli 2. Celem eksperymentu jest wyznaczenie optymalnej wartości parametru learning rate, dlatego badanie będzie polegało na sprawdzeniu szybkości uczenia w zależności od zastosowanego współczynnika uczenia.



Rysunek 29: Wykres zależności szybkości uczenia od współczynnika learning rate (S1 = 25, S2 = 2)



Rysunek 30: Wykres zależności szybkości uczenia od współczynnika learning rate (S1 = 15, S2 = 6)

Do zbadania wpływu współczynnika uczenia na szybkość uczenia sieci, wykonano 10 prób dla różnych wartości w danym rzędzie wielkości. W sumie dla jednej

architektury sieci przeprowadzono 70 eksperymentów dla rzędów wielkości:

$$\{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 10^0, 10^1\}$$

dlatego oś odciętych na rysunkach 29, 30 jest w skali logarytmicznej.

W obu przypadkach najbardziej optymalne okazały się wartości z przedziału $< 0.01, 0.1 >$ gdzie najlepszą wartością współczynnika uczenia dla sieci ($S1 = 15$, $S2 = 6$) jest $lr = 0.09$, natomiast dla sieci ($S1 = 25$, $S2 = 2$) $lr = 0.1$.

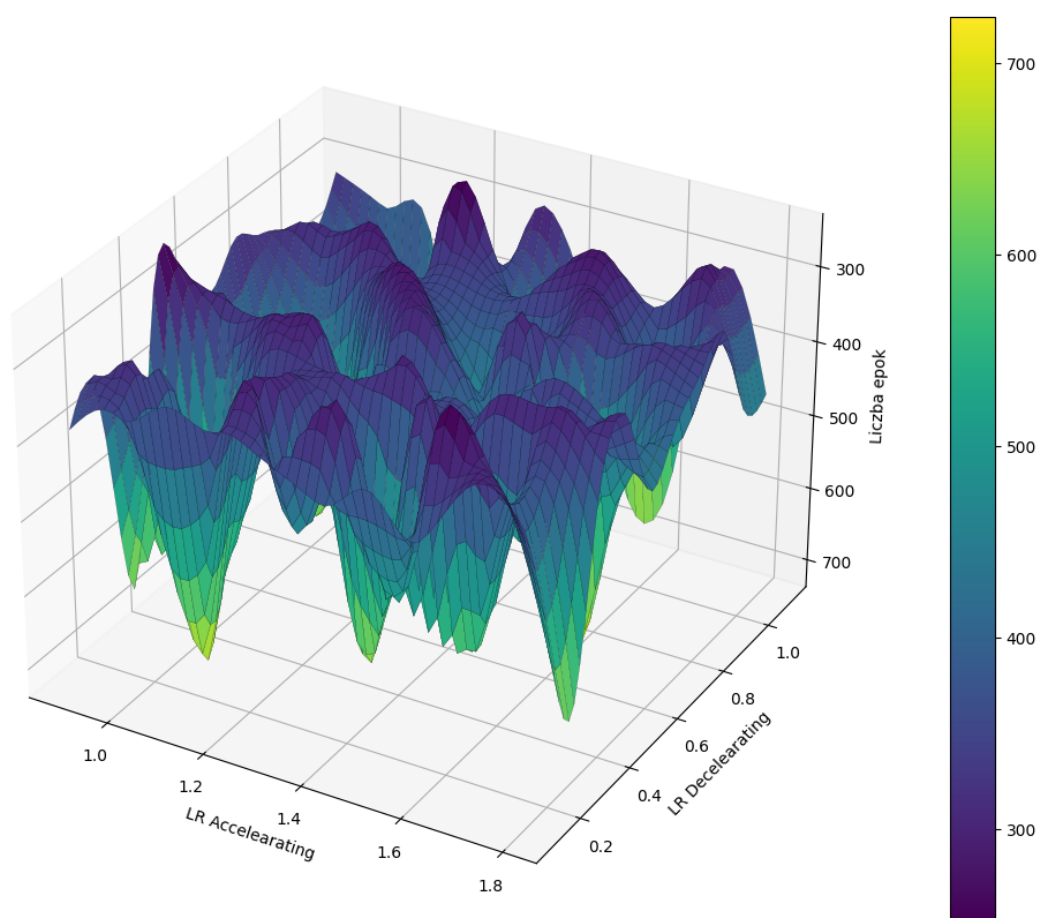
Poddając analizie otrzymane wyniki, można dojść do wniosku, że proces uczenia przebiega najefektywniej w wąskim zakresie wartości learning rate, w przypadku wartości większych od 0.1 proces uczenia dla prezentowanych sieci danych przebiegał w sposób niestabilny, dlatego nie osiągnięto zakładanego celu - $SSE < 0.25$. Natomiast dla zbyt małego learning rate proces uczenia znacznie się wydłuża, dlatego wartości learning rate poniżej 0.001 okazały się mało efektywne i nie pozwoliły osiągnąć zadanego celu mieszcząc się przy tym w założonym limicie 1000 cykli uczenia.

Warto zauważyć, że dla obu sieci w efektywnym zakresie $< 0.01, 0.1 >$ wartości współczynnika uczenia wraz ze zmniejszaniem wartości współczynnika, zwiększa się ilość cykli potrzebnych do osiągnięcia celu uczenia, co potwierdza to tezę, iż zbyt mały współczynnik uczenia może znacznie wydłużyć proces nauki [1.3.4].

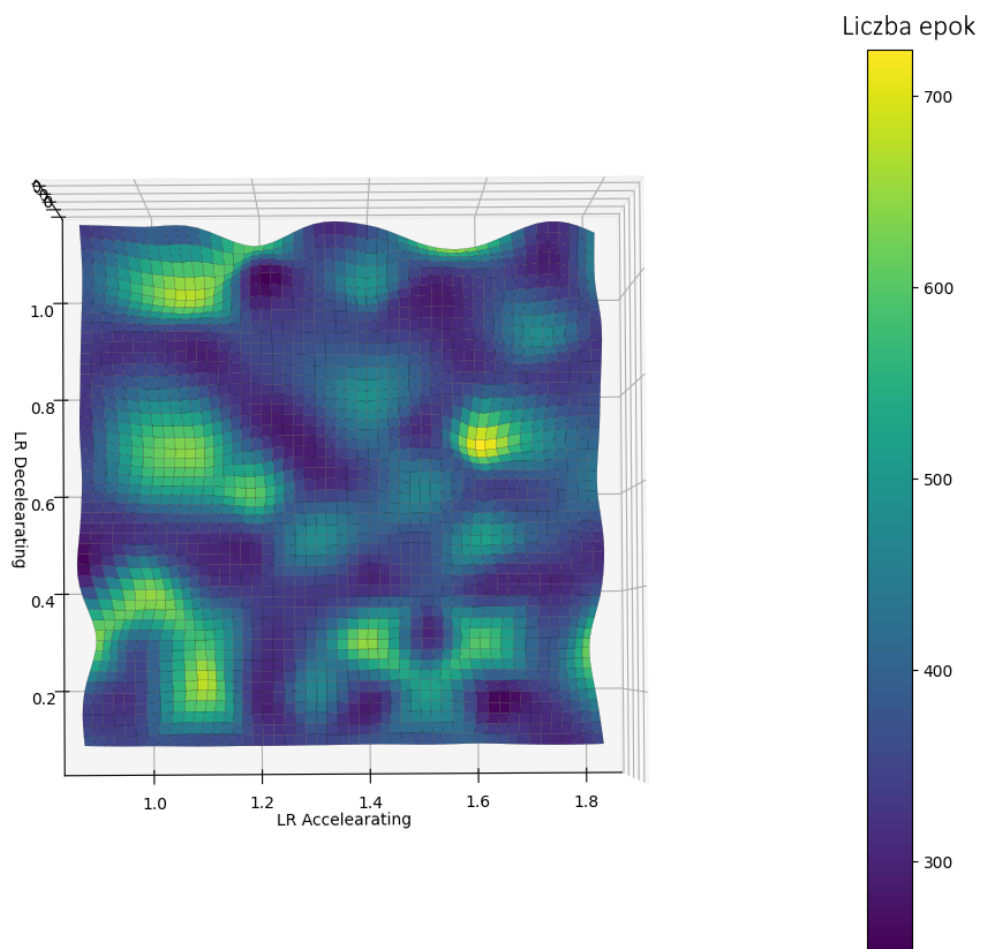
3.1.4 Badanie wpływu współczynników learning rate accelerate, decelerate

Na potrzeby eksperymentu przyjęto parametry uczenia określone w tabeli 2, zmianie poddane zostały parametry:

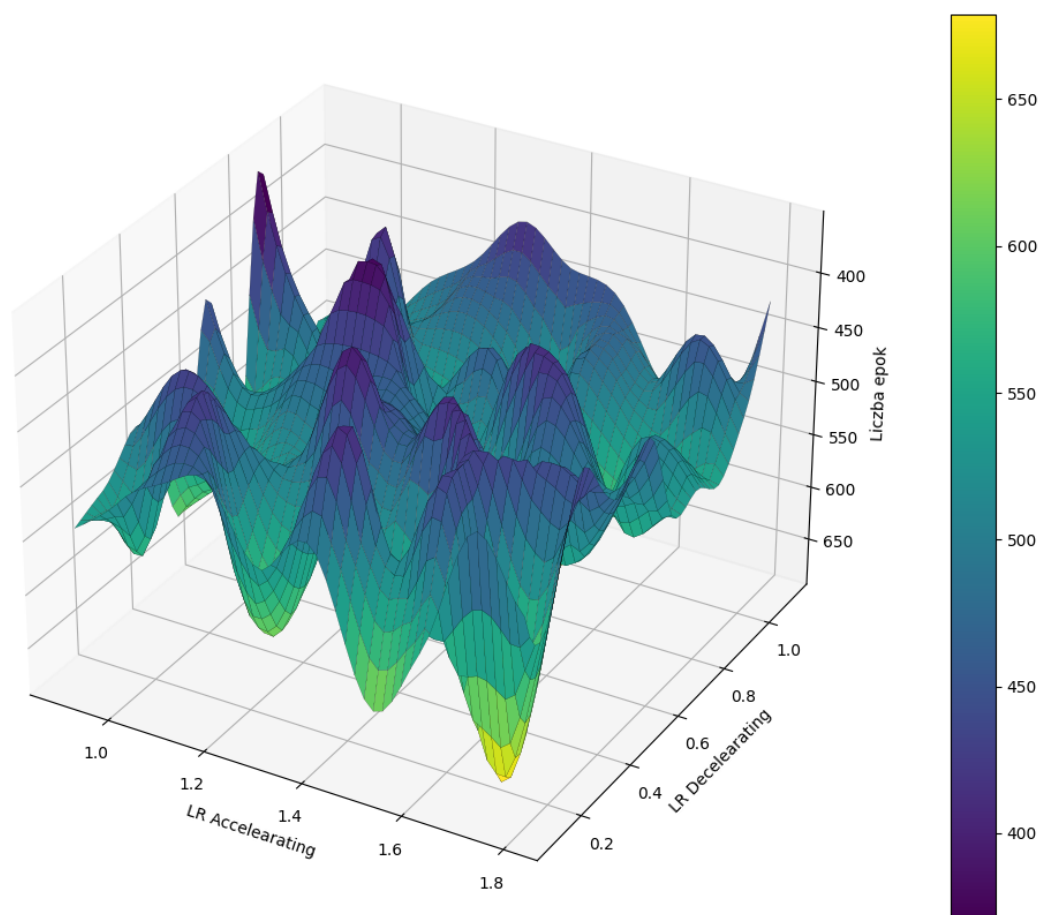
- learning rate accelerate - w przedziale $< 0.9, 1.9 >$, ze skokiem co 0.1
- learning rate decelerate - w przedziale $< 0.1, 1.1 >$, ze skokiem co 0.1



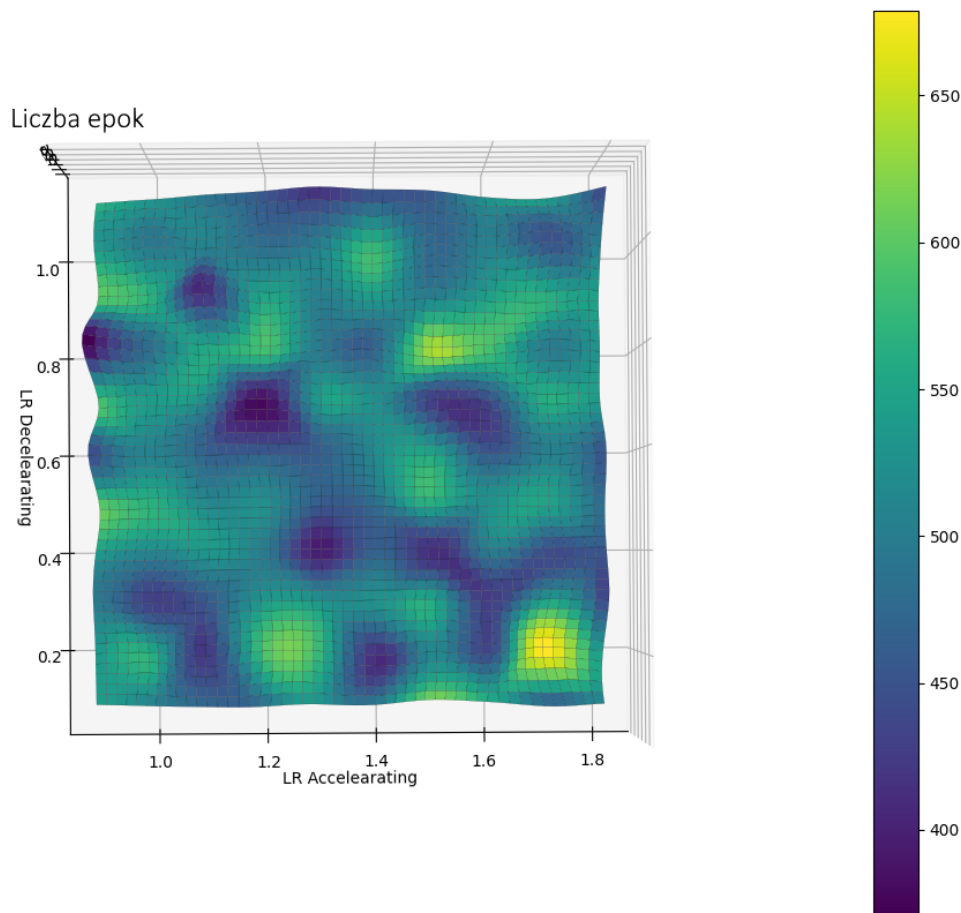
Rysunek 31: Wykres szybkości uczenia ($S1 = 25$, $S2 = 2$) od lr accelerate, lr decelerate



Rysunek 32: Wykres szybkości uczenia ($S1 = 25$, $S2 = 2$) od lr accelerate, lr decelerate



Rysunek 33: Wykres szybkości uczenia ($S1 = 15$, $S2 = 6$) od zależności lr accelerate, lr decelerate



Rysunek 34: Wykres szybkości uczenia ($S1 = 15$, $S2 = 6$) od lr accelerate, lr decelerate

Rysunki 31($S1 = 25$, $S2 = 2$), oraz 32($S1 = 15$, $S2 = 6$), przedstawiają powierzchnię funkcji liczby epok potrzebnych do nauczenia sieci w zależności od wartości parametru learning rate accelerate oraz learning rate decelerate.

Optymalne konfiguracje badanych parametrów są odmienne dla przedstawionych sieci:

- ($S1 = 25$, $S2 = 2$) - największą szybkość uczenia osiągnięto dla wartości lr accelerate = 1.2, lr decelerate = 0.9 - 257 epok. Natomiast najgorszy wynik uzyskano dla kombinacji (1.6,0.7)
- ($S1 = 15$, $S2 = 6$) - największą szybkość uczenia osiągnięto dla wartości lr

accelerate = 0.9, lr decelerate = 0.8 - 348 epok. Natomiast najgorszy wynik uzyskano dla kombinacji (1.7,0.2)

Porównując przedstawione wykresy powierzchniowe 34, 32 można dojść do wniosku, że sieć (S1=25,S2=2) posiada lepsze uwarunkowanie dla wykorzystywanego zestawu danych. Większość wartości na powierzchni jest bliskie tej optymalnej - niezależnie od kombinacji badanych hiperparametrów LR accelerate, LR decelerate sieć nadal będzie uczyć się w krótkim czasie.

3.1.5 Wnioski

Na podstawie przeprowadzonej serii eksperymentów określono zestaw hiperparametrów, dla których sieć powinna osiągnąć najlepszą wydajność [3]:

S1 (neurony warstwy I)	25
S2 (neurony warstwy II)	2
Learning rate	0.01
Learning rate accelerating	1.2
Learning rate decelerating	0.9
Error ratio	1.04
Momentum constans	0.58
Funkcja aktywacji	Bipolarna
Stosunek wielkości zbiorów	80% : 20%
Funkcja inicjalizacji wag	Losowa z ustaloną entropią
Zakres inicjalizacji wag	<-0.1,0.1>
Zakres inicjalizacji biasów	<-0.01,0.01>
Metoda uczenia	Wsadowa
Maksymalna ilość epok	1000
Oczekiwana wartość funkcji celu	$SSE < 0.25$

Tabela 3: Zestawienie hiperparametrów sieci wraz z parametrami uczenia

Przeprowadzono eksperyment mający potwierdzić postawioną hipotezę[3.1.5]. W rezultacie eksperymentu sieć osiągnęła gorszą efektywność w procesie uczenia niż w przypadku wykonywania prób dla domyślnych wartości parametrów zaproponowanych w tabeli 2. Doświadczenie pokazuje, że wszystkie hiperparametry są od siebie zależne. Modyfikując jedno jednocześnie zmianie ulega charakterystyka pozostałych hiperparametrów, natomiast określenie optymalnej kombinacji wszystkich parametrów jest zadaniem szczególnie wymagającym. Ze względu na to proces doboru hiperparametrów powinien ograniczać się do określenia optymalnej architektury sieci, pozostawiając wartości pozostałych hiperparametrów domyślne[9].

3.2 Eksperyment II - Zjawisko przeuczenia sieci

Eksperyment ma na celu zbadanie zjawiska przeuczenia sieci, wraz ze sprawdzeniem wpływu stosunku wielkości zbiorów treningowy-testowy. Tabela 4 zawiera parametry uczenia sieci. W trakcie eksperymentu zmianie ulegać będzie stosunek wielkości zbiorów.

S1 (neurony warstwy I)	6
S2 (neurony warstwy II)	3
Learning rate	0.1
Learning rate accelerating	1.05
Learning rate decelerating	0.7
Error ratio	1.04
Momentum constans	0.9
Funkcja aktywacji	Bipolarna
Stosunek wielkości zbiorów	Wartość zmienna
Funkcja inicjalizacji wag	Losowa z ustaloną entropią
Zakres inicjalizacji wag	$<-0.1,0.1>$
Zakres inicjalizacji biasów	$<-0.01,0.01>$
Metoda uczenia	Wsadowa
Maksymalna ilość epok	10000
Oczekiwana wartość funkcji celu	Brak

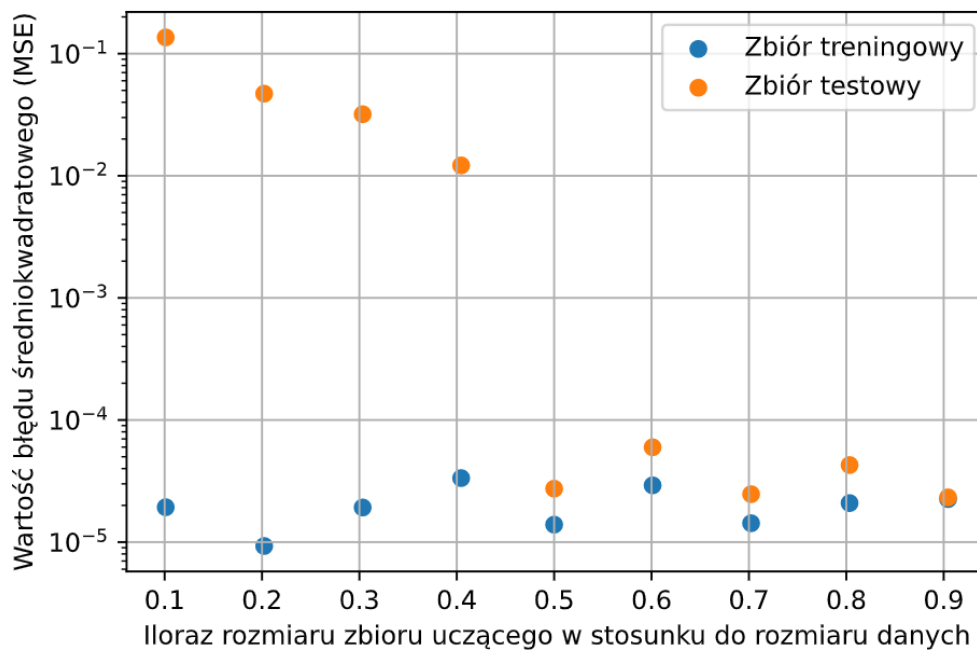
Tabela 4: Zestawienie hiperparametrów sieci wraz z parametrami uczenia

Proces uczenia dla zmiennego rozmiaru zbioru treningowego/testowego przeprowadzono w stosunkach:

- 10:90
- 20:80
- 30:70
- 40:60
- 50:50
- 60:40
- 70:30
- 80:20

- 90:10

Podczas każdej z prób zbiory były wypełniane losowymi danymi z zestawu „Wine” z zachowaniem proporcji dla każdej z klas. Rysunek 35 prezentuje zależność badanego stosunku do wartości błędu średniokwadratowego



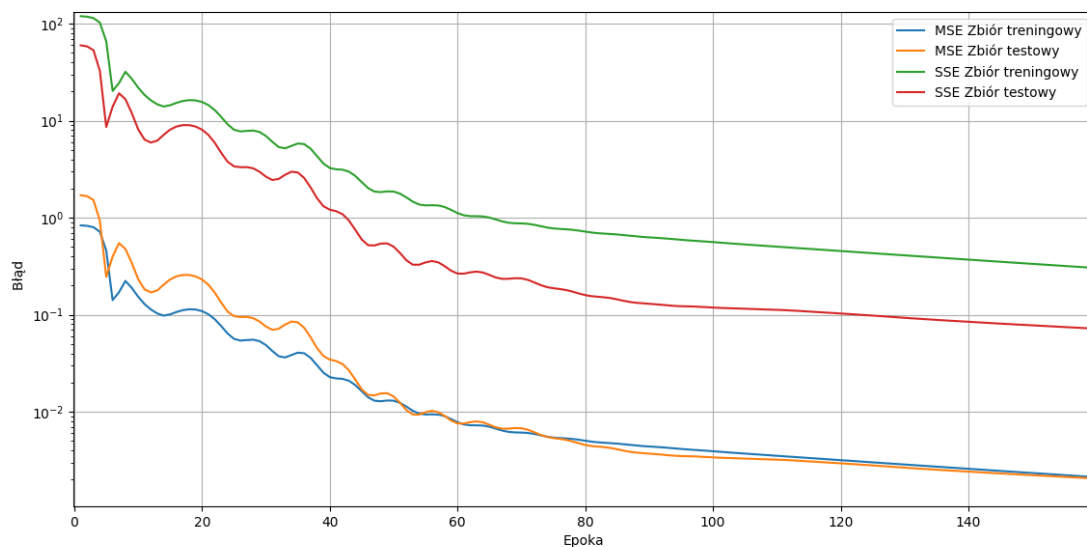
Rysunek 35: Wykres wartości MSE dla zbiorów treningowego oraz testowego

Dobrze nauczona sieć neuronowa powinna wykazywać podobną sprawność w klasyfikacji dla zestawu danych, na którym przebiegało uczenie, oraz danych na których proces uczenia nie przebiegał. Analizując uzyskane wyniki [35] można dojść do wniosku, że rozmiar danych uczących musi być conajmniej równy rozmiarowi danych testowych, aby proces nauki przebiegał poprawnie. Przy kombinacji 90:10 wartość błędu MSE dla obu zbiorów jest bliska równości, co może świadczyć o dobrym wytrenowaniu sieci - sieć jest w stanie z taką samą trafnością klasyfikować zarówno próbki biorące udział w procesie uczenia jak i również próbki, które nie brały udziału w procesie nauki. Należy jednak wziąć pod uwagę małą różnorodność zestawu danych, która może wpływać na otrzymany rezultat.

W następnej kolejności zbadano zjawisko przeuczenia sieci, tabela 5 przedstawia parametry uczenia:

S1 (neurony warstwy I)	6
S2 (neurony warstwy II)	3
Learning rate	0.1
Learning rate accelerating	1.05
Learning rate decelerating	0.7
Error ratio	1.04
Momentum constans	0.9
Funkcja aktywacji	Bipolarna
Stosunek wielkości zbiorów	80:20
Funkcja inicjalizacji wag	Algorytm Nguyen-Widrowa
Zakres inicjalizacji wag	$<-0.5,0.5>$
Zakres inicjalizacji biasów	$<-0.5,0.5>$
Metoda uczenia	Wsadowa
Maksymalna ilość epok	20000
Oczekiwana wartość funkcji celu	Brak

Tabela 5: Zestawienie hiperparametrów sieci wraz z parametrami uczenia



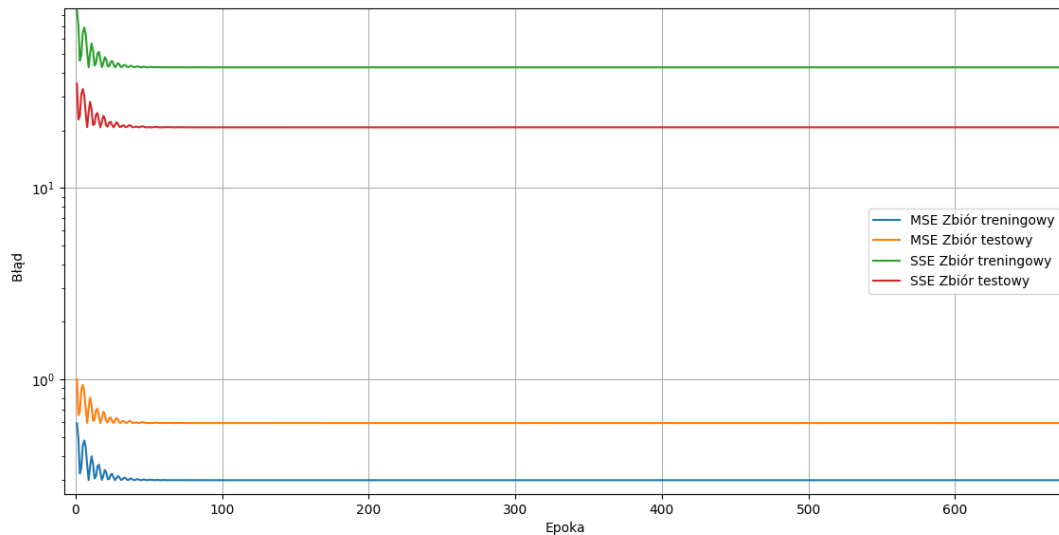
Rysunek 36: Wykres wartości MSE,SSE dla zbiorów treningowego oraz testowego

Rysunek 36 przedstawia wartość błędów MSE oraz SSE dla zbioru treningowego oraz testowego na przestrzeni epok. W początkowej fazie uczenia sieci (dla epok

< 80) wartość błędu oscyluje, natomiast w późniejszej fazie nauki wartość błędu dla obu zbiorów zaczyna się zbiegać. Moment gdy wartość błędu dla zbioru testowego jest poniżej wartości zbioru uczącego jest optymalnym wytrenowaniem sieci tj. w trakcie epok $< 70, 150 >$. Około 200 epoki błąd zbioru testowego staje się większy niż błąd zbioru uczącego, co świadczy o przetrenowaniu sieci.

Ostatnią częścią eksperymentu jest sprawdzenie wpływu normalizacji danych na proces uczenia. Plik danych wejściowych został przygotowany tak, że znormalizowane zostały jedynie identyfikatory klas, pozostałe atrybuty zostały wczytane w niezmienionej formie. Ponownie wykorzystano sieć o parametrach przedstawionych w tabeli 5. Rysunek 37 przedstawia wykres wartości funkcji celu na przestrzeni epok.

Dla pierwszych 50 epok widać wyraźny brak stabilności w procesie nauki, w kolejnych cyklach widoczny jest brak zmiany wartości błędu, co świadczy o utknięciu sieci.



Rysunek 37: Wykres błędów dla zbiorów treningowego oraz testowego dla nieznormalizowanych danych uczących

Doświadczenie pokazało istotę normalizacji danych. W przedstawionym problemie uczenia sieci neuronowej wykorzystywany zbiór danych nie jest wymagający dla sieci, jednak brak odpowiedniego przygotowania danych do procesu uczenia skutkuje brakiem pozytywnych rezultatów.

3.3 Eksperyment III - Metody aktualizacji wag

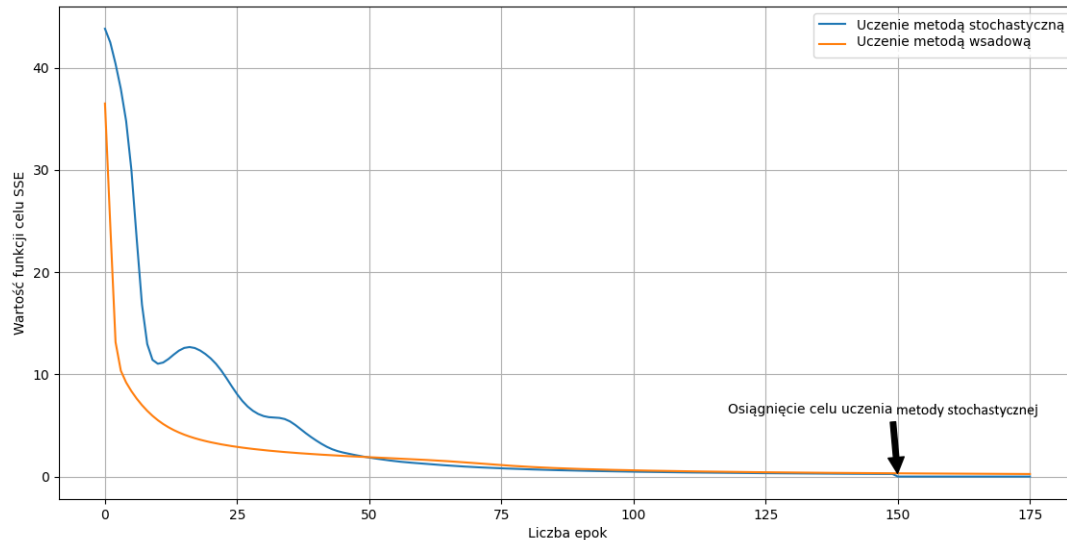
Eksperyment ma na celu sprawdzenie wpływu funkcji inicjalizującej wagi oraz zakresu losowania wag na proces uczenia. Dodatkowo w ramach eksperymentu zostanie porównana skuteczność uczenia metodą stochastyczną z metodą wsadową.

Na potrzeby eksperymentu wykorzystano sieć o następujących parametrach [6]:

S1 (neurony warstwy I)	25
S2 (neurony warstwy II)	2
Learning rate	0.01
Learning rate accelerating	1.05
Learning rate decelerating	0.7
Error ratio	1.04
Momentum constans	0.9
Funkcja aktywacji	Bipolarna
Stosunek wielkości zbiorów	80% : 20%
Funkcja inicjalizacji wag	Losowa, o rozkładzie normalnym
Zakres inicjalizacji wag	$< -0.5, 0.5 >$
Zakres inicjalizacji biasów	$< -0.5, 0.5 >$
Metoda uczenia	Wsadowa
Maksymalna ilość epok	500
Oczekiwana wartość funkcji celu	$SSE < 0.25$

Tabela 6: Zestawienie hiperparametrów sieci wraz z parametrami uczenia

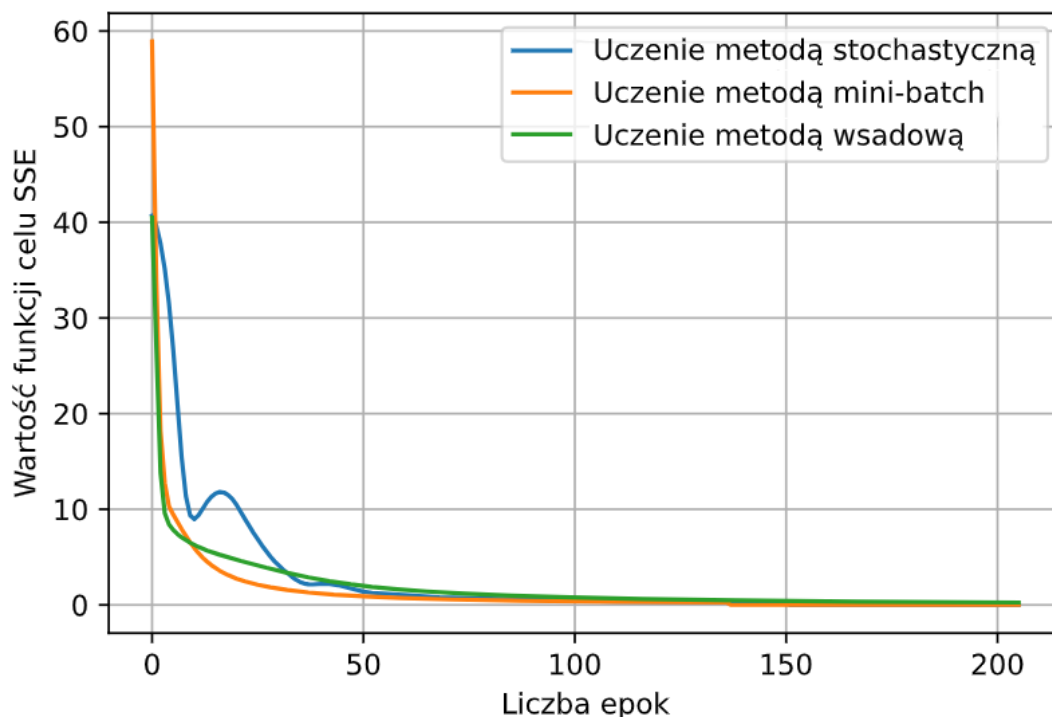
3.3.1 Porównanie metod aktualizacji wag



Rysunek 38: Wykres funkcji celu dla metody stochastycznej oraz wsadowej

Rysunek 38 zestawia ze sobą przebieg procesu uczenia sieci za pomocą metody wsadowej (linia żółta) z uczeniem metodą stochastyczną (linia niebieska). Uczenie sieci zostało zakończone pomyślnie w przypadku obu metod, jednak uczenie metodą stochastyczną zostało zakończone 25 epok wcześniej, pomimo tego, że w początkowej fazie uczenia widać wyraźne oscylacje wartości funkcji celu. Na podstawie przedstawionego wykresu [38] można dojść do wniosku, iż metoda stochastyczna jest mniej stabilna, ale pozwala na szybsze osiągnięcie zadanego celu uczenia. Jednak w przypadku gdy zbiór danych uczących będzie wprowadzał niestabilność w procesie uczenia lepsza może okazać się metoda wsadowa, ponieważ przebieg uczenia za pomocą tej metody jest stabilny, nie występują oscylacje. Zastosowanie konkretnej metody uczenia jest więc uzależnione od uwarunkowań danych, na których sieć będzie trenowana.

Metoda mini-batch jest zazwyczaj stosowana dla dużych zestawów danych, gdzie uczenie wsadowe jest zbyt czasochłonne. W celu porównania szybkości uczenia metod wykonano powtórnie próbę [39]:



Rysunek 39: Wykres funkcji celu dla metody stochastycznej, wsadowej, mini-batch(11 próbek)

Wyniki okazały się zaskakujące:

- Metoda stochastyczna - 151 epok do osiągnięcia celu uczenia
- Metoda wsadowa - 205 epok do osiągnięcia celu uczenia
- Metoda mini-batch (w jednym podcyklu uczenia pod uwagę brano 11 próbek) - 137 epok do osiągnięcia celu uczenia

Okazuje się, że dla małych zestawów danych stosowanie hybrydy metod off-line oraz on-line przynosi najlepsze rezultaty. W wyniku tej próby najgorszy wynik szybkości uzyskano metodą wsadową.

3.3.2 Porównanie metod inicjalizacji wag oraz biasów

Przed przystąpieniem do wykonania eksperymentu należy wyjaśnić na czym polega algorytm inicjalizacji wag metodą Nguyen-Widrow. Algorytm postępowania dla omawianej metody jest następujący:

1. Dla każdej warstwy wyznaczyć współczynnik θ :

$$\theta = 0.7 \cdot n^{\frac{1}{m}}$$

gdzie:

- n - ilość wejść do neuronów
- m - ilość neuronów w warstwie

2. Dla każdego neuronu w warstwie:

- (a) Wyznaczenie losowej wartości wag oraz biasu z przedziału $< -0.5, 0.5 >$ o rozkładzie normalnym
- (b) Wyznaczenie wartości współczynnika η :

$$\eta_j = \sqrt{\sum_{i=1}^n w_{ji}^2}$$

gdzie:

- j - numer neuronu w warstwie
 - n - ilość wejść neuronu
- (c) Na podstawie współczynnika η wyznaczyć nowe wagi dla neuronu według zależności:

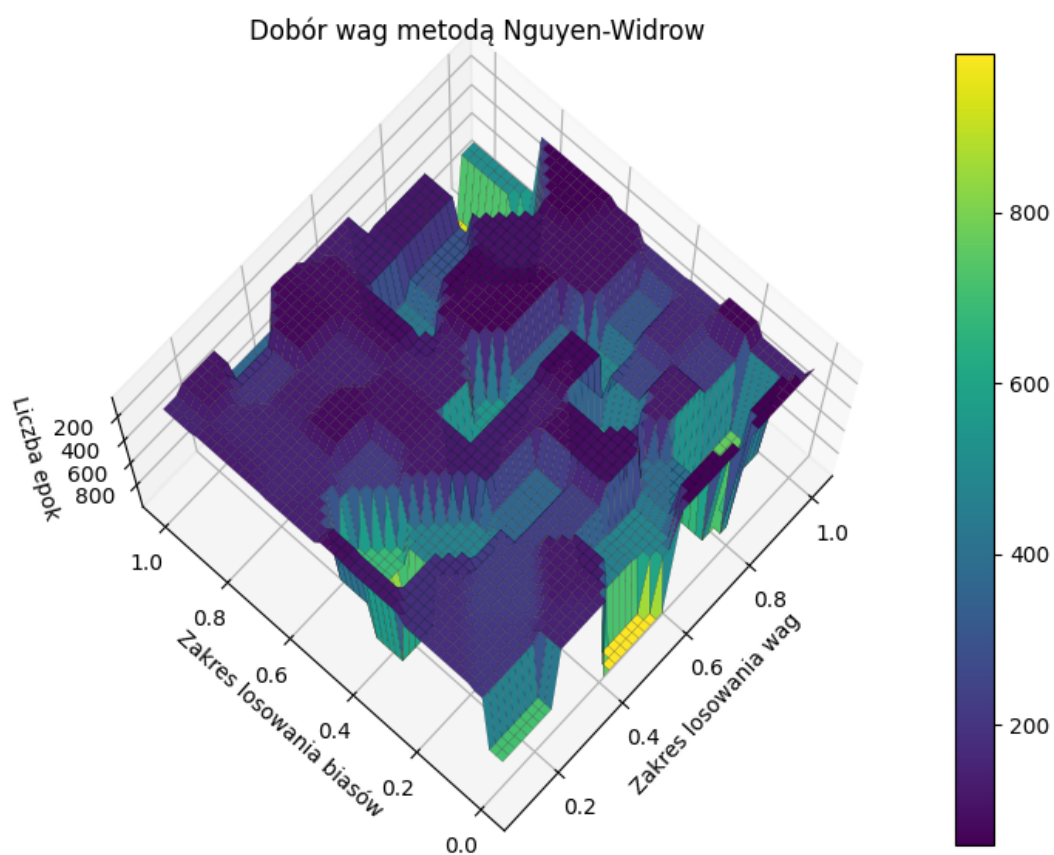
$$w_{ji} = \frac{\theta \cdot w_{ji}}{\eta}$$

gdzie:

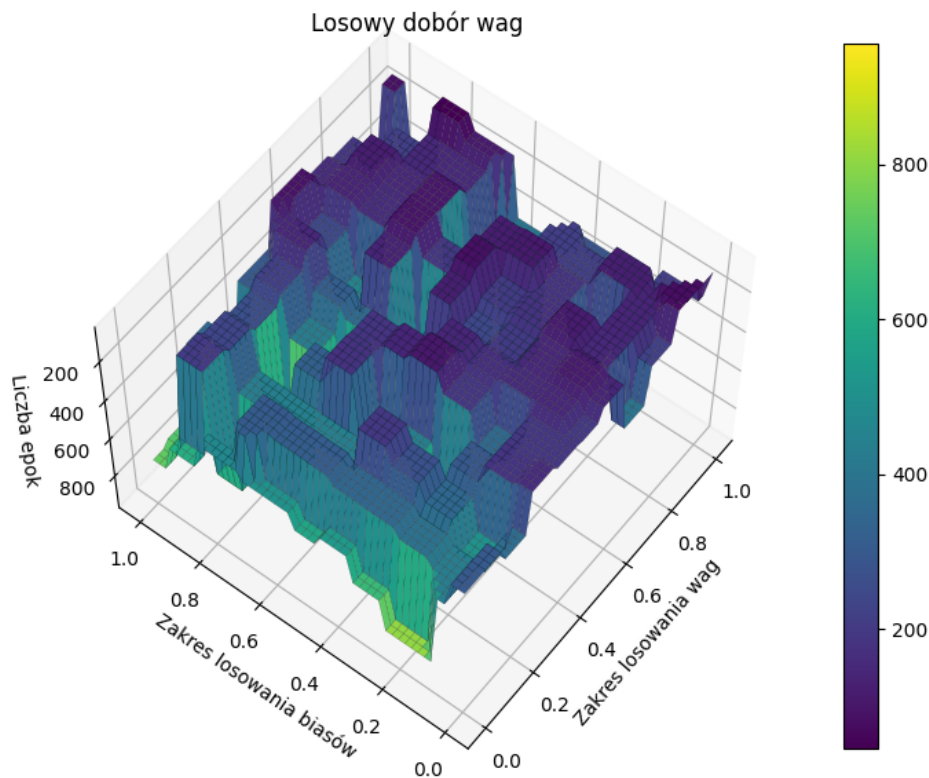
- j - numer neuronu w warstwie
- i - numer wejścia neuronu

Trafny dobór wartości początkowych wag oraz biasów ma kluczowe znaczenie dla procesu uczenia, w skrajnych przypadkach źle dobrane wartości inicjalizacyjne mogą uniemożliwić uczenie sieci. Z drugiej strony dobrze dobrane wagi wraz z biasem pozwalają skrócić czas potrzebny do osiągnięcia celu nauki. Oprócz porównania funkcji inicjalizujących wagi pod uwagę wzięto zakresy wartości spośród których wybierane są początkowe wartości wag.

Osie poziome na rysunkach 40,41 są odzwiedcieniem prawej strony symetrycznego względem zera przedziału losowania.



Rysunek 40: Wykres powierzchni dla funkcji inicjalizacji wag Nguyen-Widrow



Rysunek 41: Wykres powierzchni dla funkcji inicjalizacji losowej

Wyniki eksperymentu pokazują, że odpowiednie dobranie przedziału losowania wag może przynieść podobny rezultat bez znaczenia jaką metodą wagi zostały przypisane. W przypadku metody Nguyen-Widrow zmiany w zakresie losowania wartości początkowych pozwoliły na osiągnięcie celu uczenia po 59 epokach - zakres dla wag to $\langle -0.9, 0.9 \rangle$, dla biasów $\langle 0, 0 \rangle$. Natomiast sieć, w której początkowe wartości wag dobrano losowo, osiągnęła cel uczenia w najlepszym przypadku po 46 epokach - zakres dla wag to $\langle -0.9, 0.9 \rangle$, dla biasów $\langle -0.8, 0.8 \rangle$.

Istotny jest również rozkład wartości na przedstawionych wykresach powierzchniowych - w przypadku metody Nguyen-Widrow znaczna większość kombinacji wartości jest bliska tej najbardziej optymalnej. Metoda losowego doboru wag poprawia szybkość uczenia sieci jedynie dla niewielkiej grupy badanych kombinacji.

3.3.3 Rozszerzenie metody momentum

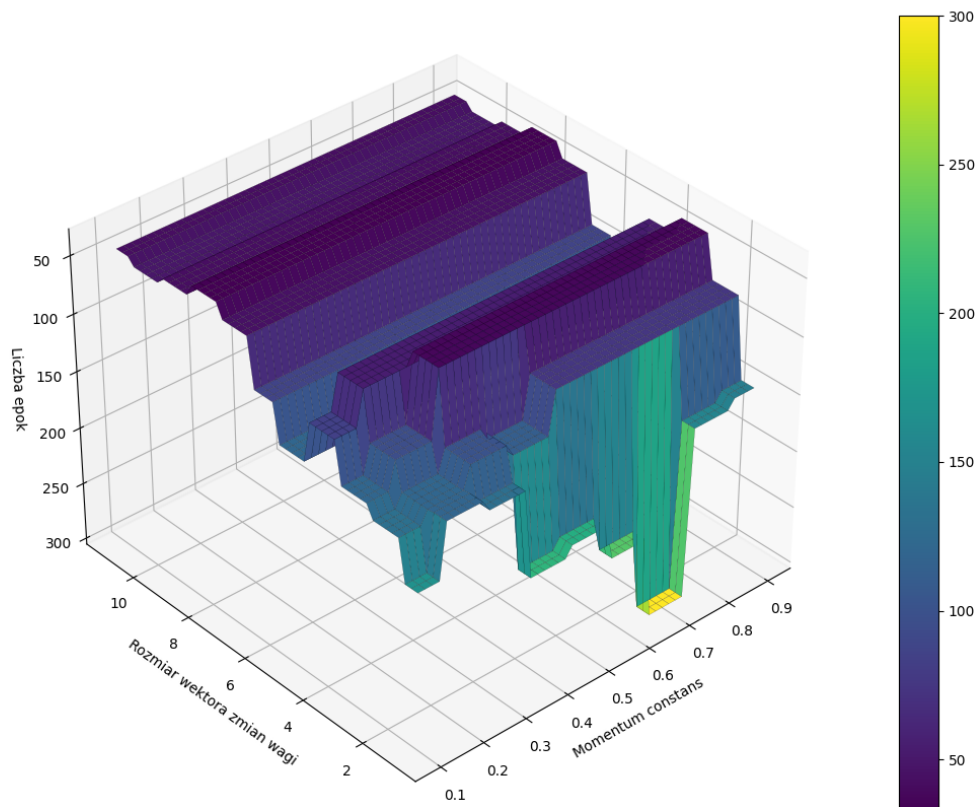
W ramach ostatniego eksperymentu sprawdzono jak zachowa się sieć, gdy metoda momentum zostanie rozszerzona o dodatkowe zmiany wag z poprzednich

iteracji.

$$w_{n+1} = w_n + \Delta w_n + \alpha \cdot \sum_{i=1}^g \left(\Delta w_{n-i} \right) \quad (18)$$

gdzie:

- w_{n+1} - nowa wartość wagi neuronu
- w_n - aktualna wartość wagi
- Δw_n - gradient wagi
- Δw_{n-i} - gradient wagi z poprzedniej i-tej wstecz iteracji wcześniej
- α - współczynnik momentum
- g - długość wektora aktualizacji wagi



Rysunek 42: Wykres powierzchni ilości epok dla zmiennego momentum constants oraz rozmiaru wektora aktualizacji wagi

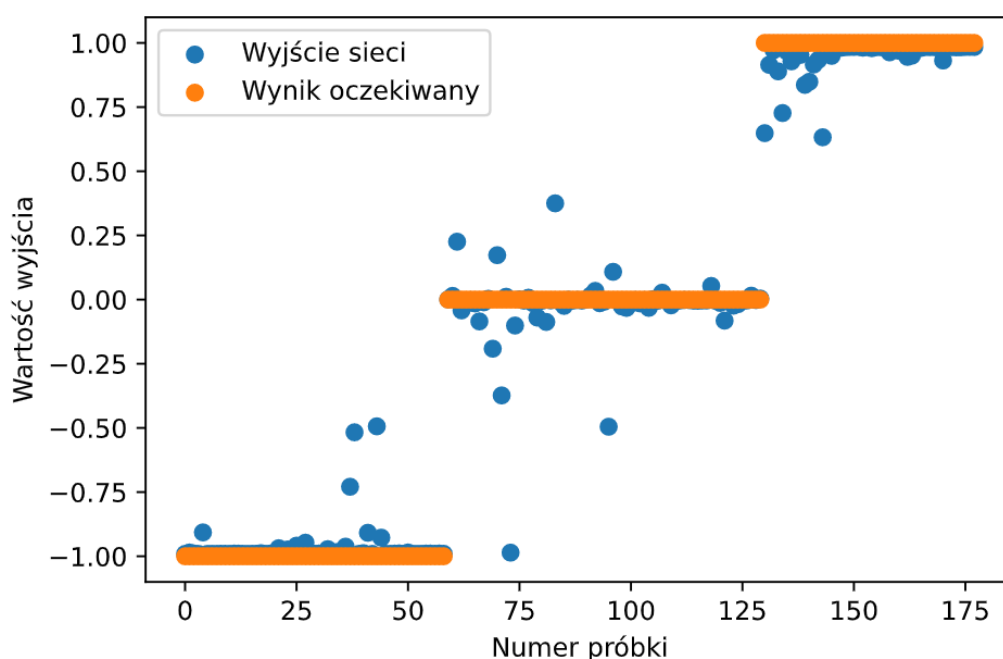
Rysunek 42 pokazuje, że zwiększanie ilości czynników, które wpływają na zmianę wagi w pewnym momencie "zagłusza" obecność współczynnika momentum - już dla 4 czynników, wartość momentum constants nie wpływa na proces uczenia. Dla wartości współczynnika g w zbiorze $\{1, 2, 3, 4\}$ wzrost współczynnika momentum wpływa na proces uczenia, co ma bezpośrednie odzwierciedlenie w kształcie powierzchni wykresu [42]. Tak więc zwiększenie ilości czynników (wartość g) może iść w parze ze zmniejszaniem wartości współczynnika momentum constants, możliwe jest wtedy osiągnięcie podobnych rezultatów. Dodanie dodatkowych czynników pędu może być pomocne w przypadku bardzo niestabilnego procesu uczenia poprzez stłumienie oscylacji.

Do rozpatrzenia pozostaje rozbudowa metody - zastosowanie średniej zmiany na przestrzeni ostatnich iteracji.

4 Wnioski

Celem projektu jest realizacja sztucznej sieci neuronowej uczonej metodą wstecznej propagacji błędów z przyspieszeniem uczenia metodą adaptacyjnego współczynnika uczenia oraz metodą momentum. Do zrealizowania sieci wykorzystano własną implementację sieci neuronowej w języku C++, co pozwoliło na kilkukrotne skrócenie czasu uczenia względem rozwiązań wykorzystujących środowisko Python lub Matlab. Dodatkowym atutem własnej implementacji sztucznej sieci neuronowej jest możliwość dowolnej ingerencji w algorytm uczenia - jak pokazano w eksperymencie III[3.3.3]. Elastyczność takiego rozwiązania otwiera szeroki wachlarz nowych możliwości w dziedzinie usprawniania procesu uczenia.

Zadaniem stworzonej sieci jest klasyfikacja danych ze zbioru „Wine” - jak zaprezentowano na rysunku 43 sieć spełnia postawione wymagania. Skuteczność klasyfikacji danych na przedstawionym wykresie wynosi 98%, co świadczy o bardzo wysokim stopniu nauczania sieci.



Rysunek 43: Wykres wartości uzyskanych na wyjściu sieci, oraz wartości oczekiwanych dla całego zbioru danych uczących

W każdym z wykonywanych eksperymentów sieć, która osiągnęła zadany poziom nauczania, osiągała skuteczność klasyfikacji na poziomie $> 98\%$.

Literatura

- [1] Wine Data Set - Machine Learning Repository [dostęp 20.05.2021].
- [2] Classification of three wines from physical and chemical attributes [dostęp 20.05.2021].
- [3] Stanisław Osowski. *Sieci neuronowe do przetwarzania informacji*. Oficyna Wydawnicza Politechniki Warszawskiej, 2006.
- [4] David B. Fogel James M. Keller, Derong Liu. *Fundamentals Of Computational Intelligence*. John Wiley and Sons Inc., 2016.
- [5] Diagram of an artificial neural network [dostęp 20.05.2021].
- [6] Hitoshi Iba Nikolay Y. Nikolaev. *Adaptive Learning od Polynominal Networks*. Springer, 2006.
- [7] The Math Behind Gradient Descent Explained [dostęp 20.05.2021].
- [8] prof. dr hab. inż. Jacek Kluska. *Metoda "delta" uczenia wielowarstwowych sieci neuronowych za pomocą wstecznej propagacji błędu*. Politechnika Rzeszowska, 2021.
- [9] Matlab - traingdx [dostęp 20.05.2021].
- [10] Repozytorium z kodem źródłowym własnej implementacji sieci neuronowej [dostęp 20.05.2021].
- [11] Bernard Widrow Derrick Nguyen. *Improving the learning speed of 2-layer neural networks by choosing initial values of the adaptive weights*. Proceedings of the International Joint Conference on Neural Networks, 1990.